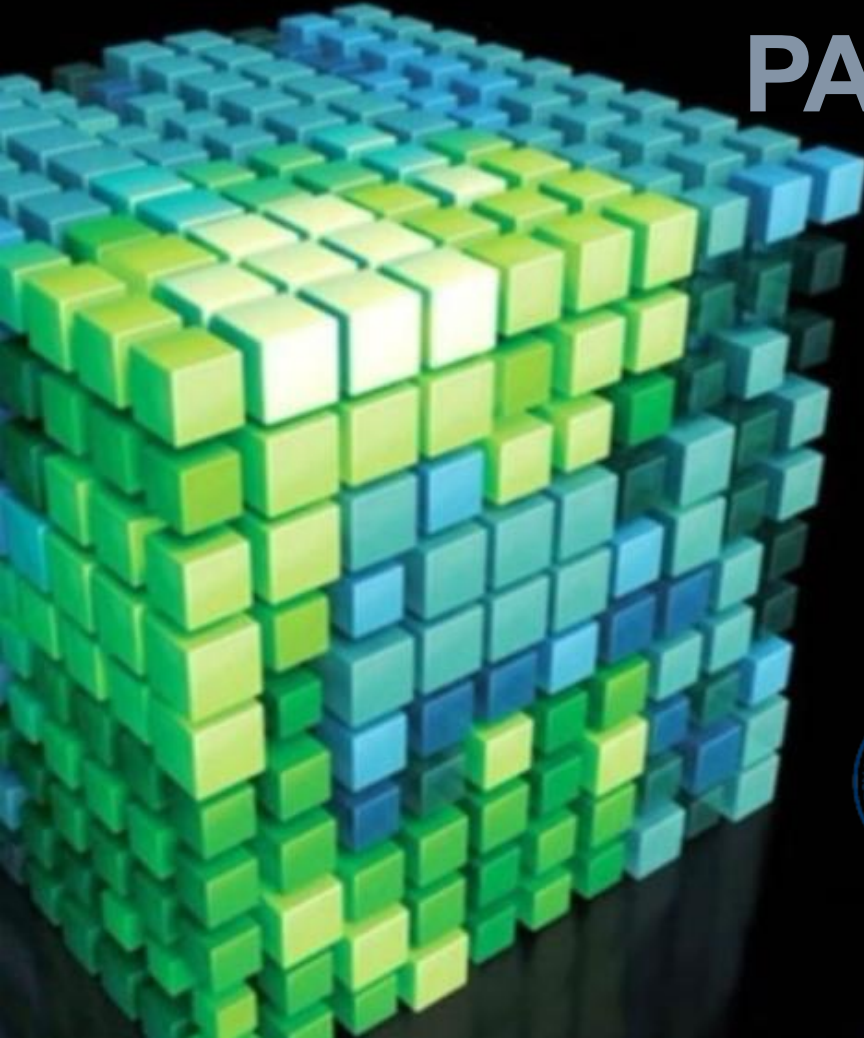




PARALELNI SISTEMI: **CUDA**



MSc Aleksandra Stojnev
Prof. Dr. Natalija Stojanović

Literatura

- [1] David Kirk, Wen-mei Hwu: *Programming Massively Parallel Processors: A Hands-on Approach*, Morgan Kaufmann
 - [2] Jason Sanders, Edward Kandrot: *CUDA by example: an introduction to general-purpose GPU programming*, Addison-Wesley Professional.
- ❖ Obratite pažnju na beleške ispod slajdova! U njima se nalaze dodatna detaljnija objašnjenja slajdova i/ili reference na poglavlja iz [1] i [2], u cilju boljeg razumevanja.

CUDA - Compute Unified Device Architecture

- Arhitektura za upravljanje izračunavanjem opšte namene na grafičkim procesorskim jedinicama, dostupna na NVIDIA grafičkim karticama serije 8000 i novije
- Dolazi sa softverskim okruženjem koje omogućava developerima da koriste C kao viši programski jezik.
- Podržani su i drugi jezici, API i “directives-based” pristupi



CUDA komponente

- Driver
 - Low-level softver koji kontroliše grafičku karticu
- Toolkit
 - Nvcc CUDA kompajler
 - Nsight IDE plugin za Eclipse ili Visual Studio
 - Alati za profajliranje i debugging
 - Različite biblioteke
- SDK
 - Mnogo primera
 - Error-checking
 - Zvanično nije podržan od NVIDIA
 - Skoro pa bez dokumentacije

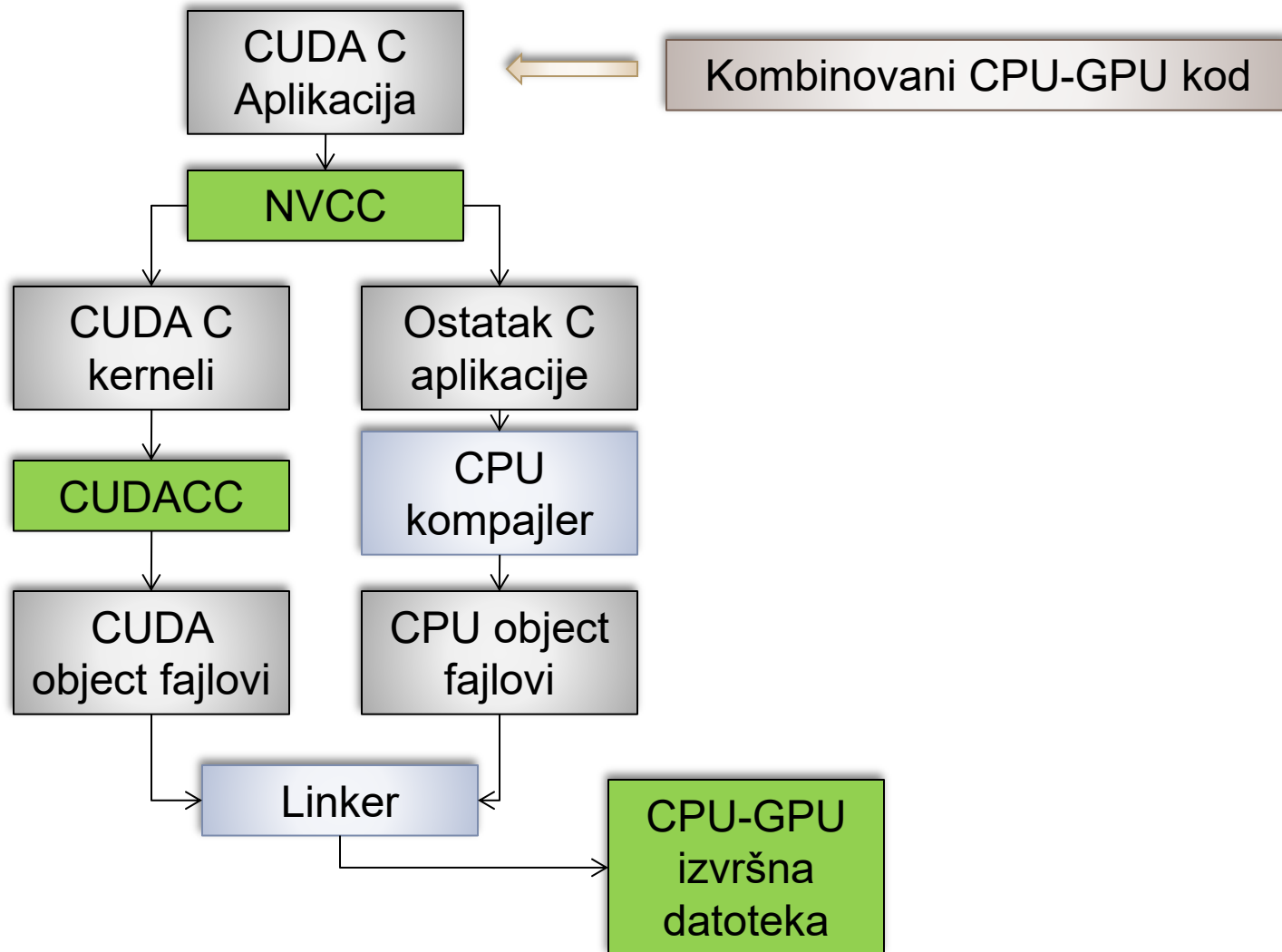
CUDA program

- CUDA program čine integrisani delovi koda za centralni i grafički procesor
 - Izvorni kod sadrži CUDA ekstenzije kojima se specificira koji delovi koda se kako i gde izvršavaju
- Prevođenje CUDA programa zahteva i prevodioca koji je u stanju da generiše kod koji se izvršava na centralnom procesoru i kod koji se izvršava na grafičkom procesoru

Prevođenje CUDA programa (1)

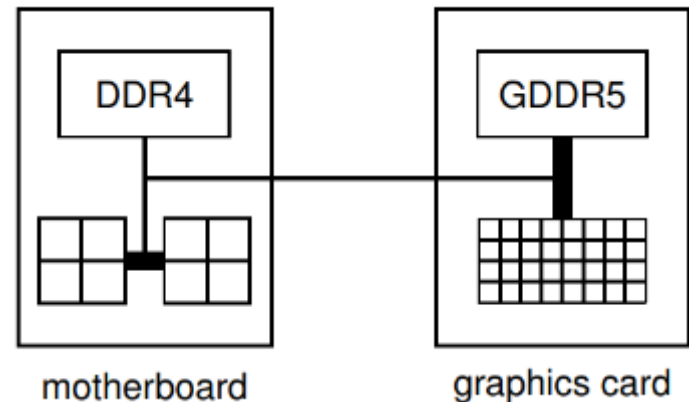
- Bilo koji izvorni kod koji sadrži CUDA ekstenzije mora se prevesti pomoću nvcc prevodioca
- NVCC je prevodilac-omotač (compiler driver)
 - Radi tako što poziva sve nepohodne alate i prevodioce
 - cudacc, g++, cl, ...
- Izlazi NVCC prevodioca su:
 - C kod koji se izvršava na strani domaćina (CPU kod) i koji se mora dalje prevesti odgovarajućim prevodiocem
 - PTX (Parallel Thread eXecution) kod
 - Predstavlja neku vrstu međukoda za grafički procesor
- Bilo koji program koji sadrži CUDA pozive, zahteva sledeće dve dinamičke biblioteke:
 - CUDA runtime biblioteku (**cuda**)
 - CUDA core biblioteku (**cuda**)

Prevođenje CUDA programa (2)



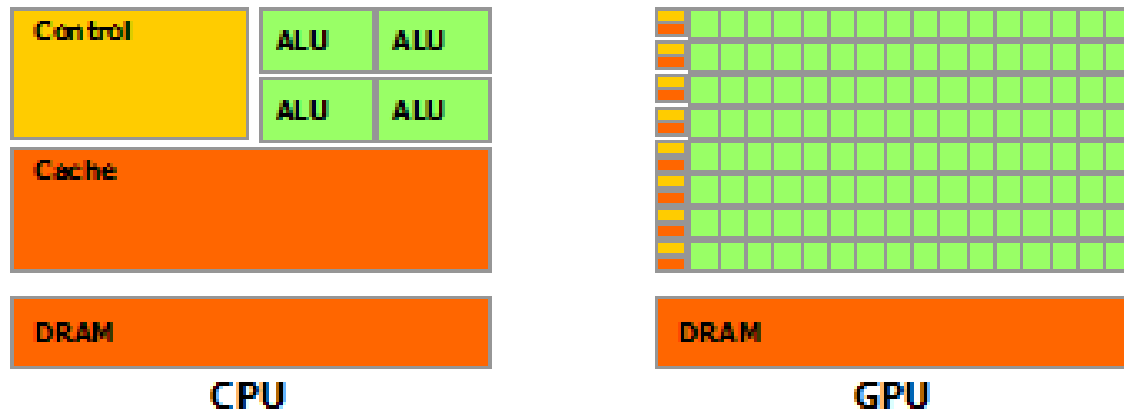
Hardverski pogled na GPU (1)

- *Massively-Parallel Many Core Architecture*
- Podrazumeva veliki broj *streaming* multiprocesora (SM) koji sami po sebi nisu preterano moćni, ali kombinovani sa dobro napisanim paralelnim kodom imaju izuzetno veliku moć računanja



CPU vs GPU (2)

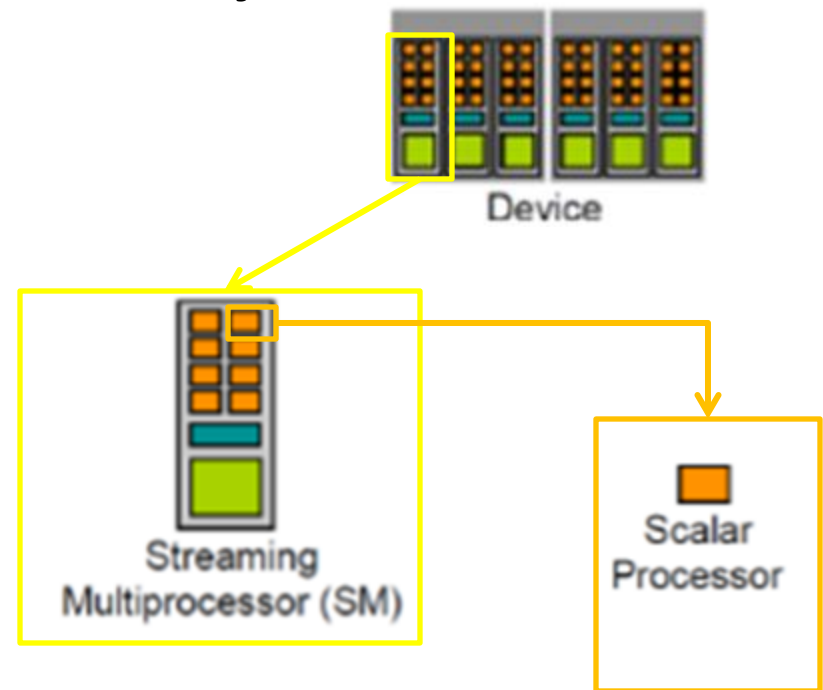
- Fundamentalna razlika između CPU i GPU je u njihovom dizajnu:



- CPU je orijentisan ka tradicionalnom izvršenju poslova, a GPU ka obradi podataka
- Kod GPU mnogo više tranzistora je namenjeno obradi podataka nego keširanju i kontroli toka

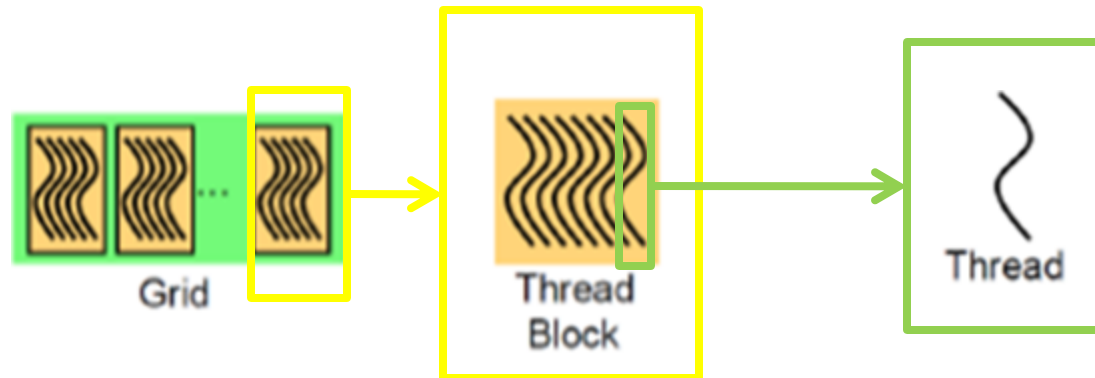
Hardverski pogled na GPU (2)

- **Device** = GPU = skup multiprocesora
- **Multiprocesor** (streaming multiprocessor - SM) = skup procesora & deljive memorije



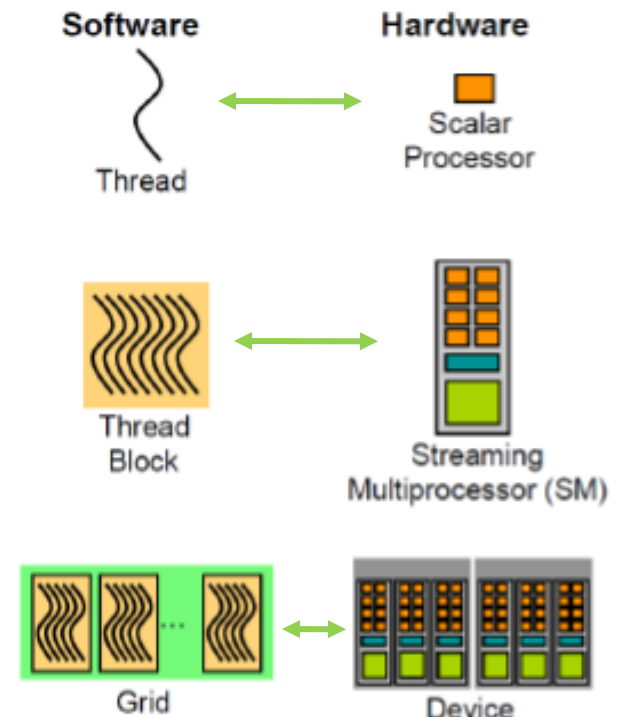
Softverski pogled na GPU (1)

- **Kernel** = Program koji se izvršava na GPU
- **Grid** (Rešetka) = Skup blokova niti koje izvršavaju kernel
- **Blok niti** = Grupa SIMD niti koje izvršavaju kernel i mogu komunicirati preko deljive memorije
- Kernel se izvršava kao grid blokova niti



Preslikavanje HW → SW

- **Grid** → **GPU**: Ceo grid je opslužen od strane GPU čipa
- **Blok** → **SM**: Svaki SM je odgovoran za opsluživanje jednog ili više blokova. Blok niti se nikada ne deli između različitih SM-a
- **Nit** → **SP**: Svaki SM je podeljen na više SP, gde je svaki od njih odgovoran za pojedinačnu nit iz bloka



HW → SW

- Blokovi grida (rešetke) se enumerišu i distribuiraju multiprocessorima
- Niti unutar jednog bloka niti se izvršavaju konkurentno na jednom multiprocessoru
- Više blokova niti se mogu izvršavati istovremeno na jednom multiprocessoru
- Kako se blokovi niti završavaju, novi blokovi se pokreću na oslobođenim multiprocessorima
- **SM je dizajniran da konkurentno izvršava na hiljade niti**
- Svaka nit se izvršava na skalarnom procesoru (SP)
- Instrukcije se izvršavaju redom: nema predikcije grananja i nema spekulativnog izvršavanja

Softverski pogled (2)

- Na visokom nivou, master proces koji se izvršava na CPU vrši sledeće:
 1. Inicijalizacija kartice
 2. Alokacija memorije na hostu i device-u
 3. Kopiranje podataka iz memorije hosta u memoriju device-a
 4. Pokretanje većeg broja blokova na device-u
 5. Kopiranje podataka iz memorije device-a u memoriju hosta
 6. Ponavljanje 3-5 po potrebi
 7. Oslobađanje memorije i završavanje programa

Softverski pogled (3)

- Na nižem nivou (GPU):
 1. Svaki blok se izvršava na SM
 2. Ako je broj blokova veći od broja SM, na svakom SM će se izvršavati više od jednog bloka ako ima dovoljno registara i deljive memorije, dok će ostali čekati u redu i biti izvršeni kasnije
 3. Sve niti u okviru jednog bloka mogu pristupati lokalnoj deljivoj memoriji, ali ne mogu videti šta drugi blokovi rade (čak i kada su na istom SM)
 4. Ne postoje garancije za redosled kojim se blokovi izvršavaju

Programski model (1)

- Grafički procesor se posmatra kao koprocesor (uređaj, compute device) u odnosu na centralni procesor (domaćin, host)
 - Izvršava računski intenzivan deo aplikacije
 - Izvršava jako veliki broj niti u paraleli
 - Posедуje svoju sopstvenu DRAM memoriju
- Deo aplikacije koji vrši obradu nad podacima izvršava se u vidu jezgra (kernel) koristeći veliki broj niti
 - GPU niti su lake (lightweight)
 - Imaju veoma mali režijski trošak prilikom stvaranja
 - GPU-u su potrebne hiljade niti za punu efikasnost
 - Višejezgarom procesoru je potrebno samo nekoliko

Programski model (2)

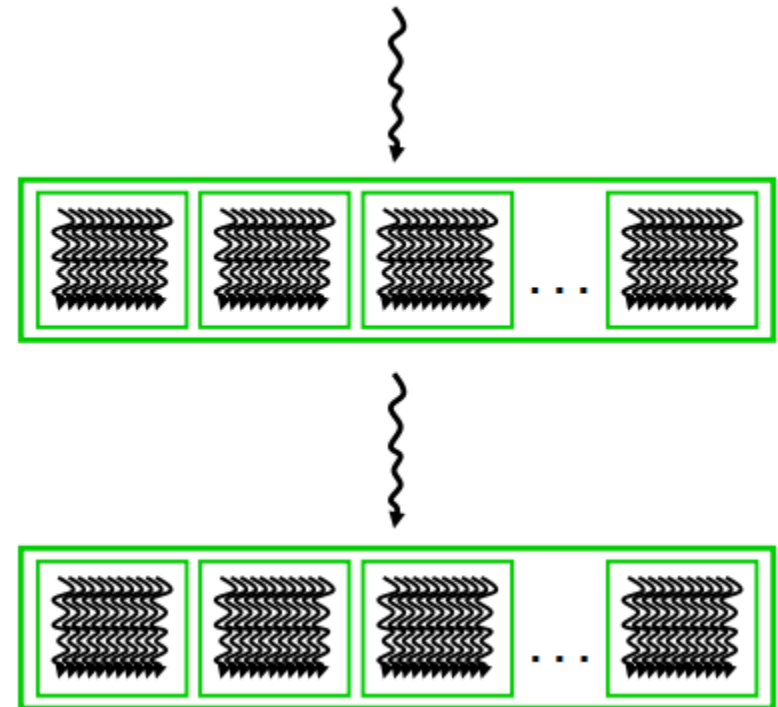
- CUDA program čine integrisani delovi koda za centralni i grafički procesor

Serial Code (host)

Parallel Kernel (device)
`KernelA<<< nBlk, nTid >>>(args);`

Serial Code (host)

Parallel Kernel (device)
`KernelB<<< nBlk, nTid >>>(args);`



Programski model (3)

- Najjednostavnija forma kernela:

```
kernel_routine<<<gridDim, blockDim>>>(args);
```

- *gridDim* – broj blokova (veličina “grida”)
- *blockDim* – broj niti unutar svakog bloka (veličina bloka)
- *args* – limitirani broj argumenata, najčešće pokazivača na nizove na GPU, i konstante koje se kopiraju po vrednosti
- Generalnija forma dozvoljava da *gridDim* i *blockDim* budu 2D ili 3D kako bi se pojednostavilo pisanje programa

Programski model (4)

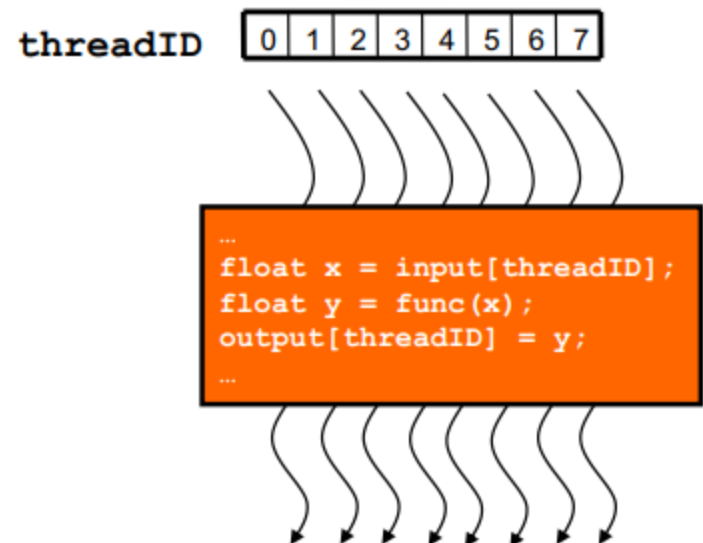
- Na nižem nivou, kada se pokrene jedan blok na SM, njega izvršava veći broj niti koje imaju pristup:
 - Promenljivama koje su prosleđene kao argumenti
 - Pokazivačima na nizove na device-u
 - Globalnim konstantama u memoriji device-a
 - Deljivoj memoriji i privatnim registrima/lokalnim promenljivama
 - Specijalnim promenljivama:
 - *gridDim* – veličina (dimenzije) grida blokova (broj blokova u gridu)
 - *blockDim* – veličina (dimenzije) svakog od blokova (broj niti u bloku)
 - *blockIdx* – indeks (ili 2D/3D indeksi) bloka
 - *threadIdx* – indeks (ili 2D/3D indeksi) niti
 - *warpSize* – uvek 32 za sada, ali može biti promenjen u budućnosti

Programski model (5)

- Kernel kod:
 - Napisan je iz ugla jedne niti
 - Razlikuje se od OpenMP multithreading-a
 - Sličan je modelu koji ima MPI gde se “rank” koristi za identifikaciju MPI procesa
 - Sve lokalne promenljive su privatne za nit
 - Potrebno je obratiti pažnju na to gde svaka od promenljivih “živi”:
 - Svaka operacija koja uključuje podatke koji se nalaze u memoriji uređaja zahteva da isti budu prebačeni u/iz GPU registara
 - Često je bolje kopirati vrednost u lokalnu registarsku promenljivu

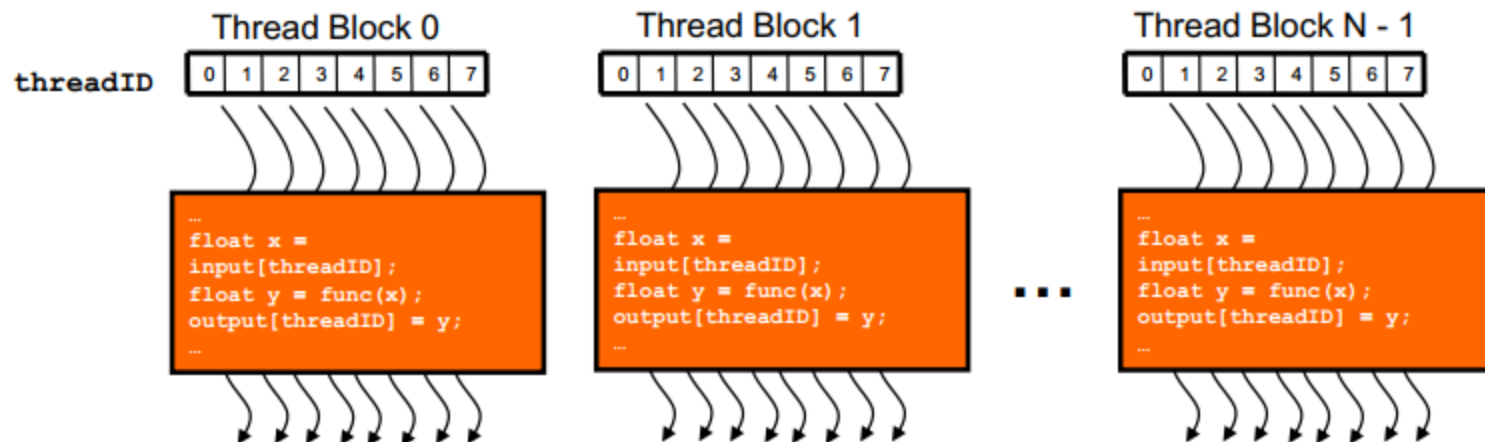
Izvršni model (1)

- CUDA jezgro se izvršava pomoću niza niti raspoređenih u odgovarajuću rešetku (grid)
 - Sve niti izvršavaju isti kod
 - SIMD/SPMD/SIMT model izvršavanja
 - Svaka nit ima jedinstveni identifikator (indeks) koji koristi da bi vršila pristup memoriji i donosila odluke



Izvršni model (2)

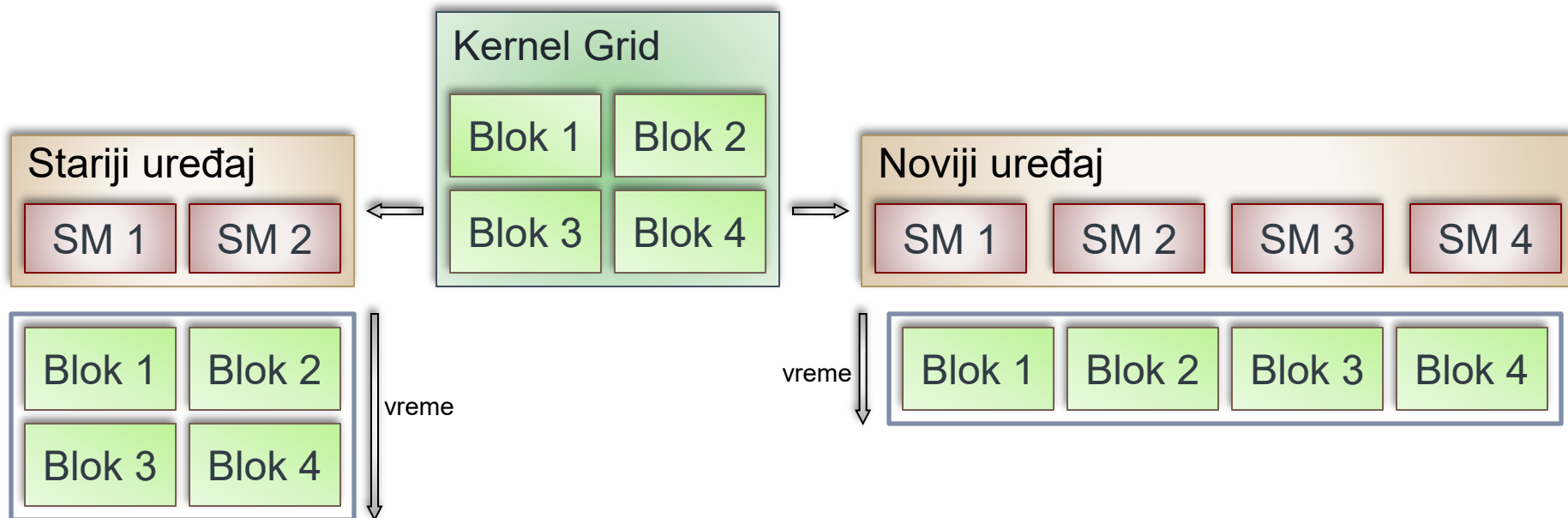
- Niti unutar rešetke su podeljene u nezavisne blokove
 - Svaki blok ima jedinstven identifikator unutar rešetke
 - Niti unutar istog bloka mogu da sarađuju
 - Koristeći sinhronizaciju, atomske operacije i deljenu memoriju
 - Niti iz različitih blokova ne mogu da sarađuju
- Na ovaj način se omogućava transparentno skaliranje na bilo koji broj procesora



Skalabilnost

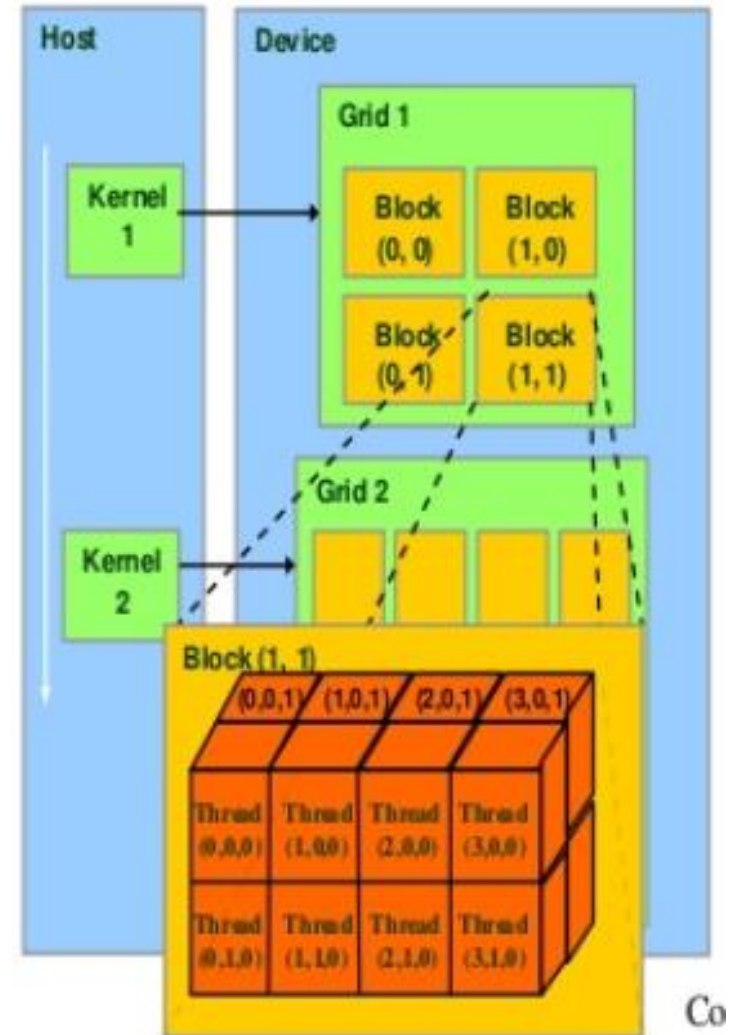
- Blokovi mogu izvršiti u bilo kojem redu u odnosu na druge blokove.

Novi uređaj može izvršiti više blokova paralelno obezbeđujući bolje performanse.



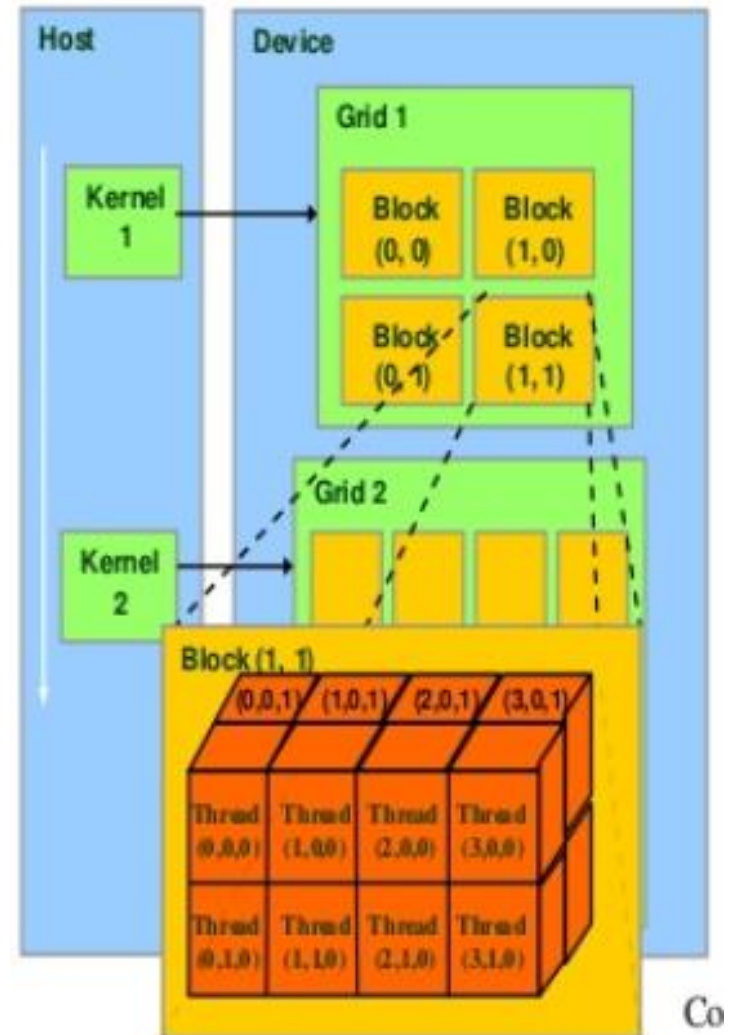
Izvršni model (3)

- Jezgro se konfigurira prilikom svakog poziva
 - Zadaju se dimenzije bloka i rešetke
 - Blok i rešetka mogu biti višedimenzionalni
 - 1D, 2D ili 3D
- Niti i blokovi imaju identifikatore (indekse)
 - Tako da mogu da odluče nad kojim podacima da rade



Izvršni model (4)

- Za svaki blok se može odrediti indeks unutar rešetke
 - Block ID: 1D, 2D, 3D
 - ***blockIdx*** promenljiva
- Za svaku nit se može odrediti indeks unutar bloka
 - Thread ID: 1D, 2D, 3D
 - ***threadIdx*** promenljiva
- Pojednostavljuje pristup memoriji pri obradi višedimenzionalnih struktura
 - Obrada slika i sl.



Razvoj CUDA aplikacija

Razvoj CUDA aplikacija

- Alokacija memorije i prenos podataka između GPU i CPU
 - Alokacija memorije
 - Memorijski transferi
- CUDA ekstenzije
 - Karakteristike funkcija koje se izvršavaju na hostu i uređaju
 - global, host & device funkcije
 - Izgled CUDA aplikacije
 - Osnovni koraci

Alokacija memorije i prenos podataka

- Alokacija memorije na strani domaćina se vrši statički ili standardnim C pozivima
- Alokacija memorije na uređaju se vrši putem odgovarajućih poziva API funkcija
- **cudaMalloc()**
 - Alocira objekat u globalnoj memoriji uređaja
 - Zahteva dva parametra
 - Adresu pointera na alocirani objekat
 - Veličinu alociranog objekta u bajtovima
 - `cudaMalloc((void **)&d_x, nbytes);`
- **cudaFree()**
 - Oslobađa objekat iz memorije uređaja
 - Zahteva pokazivač na objekat

Memorijski transferi

- Za prenos podataka između domaćina i uređaja, kao i unutar samog uređaja postoje odgovarajući pozivi
- `cudaMemcpy()`
 - Obavlja memorijske transfere
 - Zahteva četiri parametra:
 - Pokazivač na odredište
 - Pokazivač na izvor
 - Veličinu podataka koji se prenose u bajtovima
 - Tip transfera
 - `cudaMemcpy(h_x, d_x, nbytes, cudaMemcpyDeviceToHost);`
- Tipovi transfera
 - Host to Host(`cudaMemcpyHostToHost`)
 - Host to Device(`cudaMemcpyHostToDevice`)
 - Device to Host(`cudaMemcpyDeviceToHost`)
 - Device to Device(`cudaMemcpyDeviceToDevice`)
- Poziv `cudaMemcpy()` je sinhron
 - Kontrola se vraća CPU nakon što se kopiranje završi
 - Kopiranje startuje nakon što svi prethodni CUDA pozivi budu kompletiran

CUDA ekstenzije - funkcije

- CUDA program čine integrirani delovi koda za centralni i grafički procesor
- Kako prepoznati koja se funkcija gde izvršava?
 - Kvalifikator **__global__** označava kernel funkcije
 - Kvalifikator **__host__** označava funkcije koje se izvršavaju samo na strani domaćina
 - Kvalifikator **__device__** označava funkcije koje se izvršavaju samo na strani uređaja

	Izvršava:	Poziva:
__device__ float deviceFunc()	uređaj	uređaj
__global__ void kernelFunc()	uređaj	domaćin
__host__ float hostFunc()	domaćin	domaćin

CUDA funkcije – kernel (1)

- Funkcije kernela imaju sledeće osobine
 - Definišu se kvalifikatorom **__global__**
 - Moraju biti **void funkcije**
 - Parametri jezgra mogu biti skalarni podaci ili pokazivači na podatke alocirane na uređaju

__global__

void vecAdd(int *devA, int *devB, int *devC, int n);

CUDA funkcije – kernel (2)

- Kernel mora biti pozvan pomoću odgovarajuće izvršne konfiguracije
 - Zadaje se pomoću sintaksne ekstenzije jezika C, pomoću trostrukih zagrada <<< i >>>
`myKernel<<< n, m >>>(arg1, ... );`
- Parametri n i m definišu organizaciju blokova niti na nivou grida i niti na nivou bloka
- Postoje još dva opciona parametra
 - Za eksplicitno rezervisanje deljene memorije na nivou bloka
 - Za upravljanje tokovima (streams)
- Svaki poziv jezgru je asinhron
 - Kontrola se odmah vraća centralnom procesoru
 - Kernel se izvršava nakon što su svi prethodni CUDA pozivi kompletirani

CUDA funkcije - ograničenja

- **__device__** funkcijama se ne može uzeti adresa
 - One se najčešće implementiraju kao inline funkcije
- Za funkcije koje se izvršavaju na uređaju:
 - Ograničeno dozvoljena rekurzija
 - Hardversko ograničenje – stek u deljenoj memoriji
 - Od Fermi arhitekture GPU-ova
- Nije dozvoljeno deklarisanje statičkih promenljivih unutar funkcije
- Nisu dozvoljene funkcije sa varijabilnim brojem argumenata
 - Funkcije poput printf(...)

Say Hello to CUDA (1)

```
int main( void )  
{  
    printf( "Hello, World!\n" );  
    return 0;  
}
```

Say Hello to CUDA (2)

```
#include "book.h"

__global__ void kernel( void ) {}

int main( void )
{
    kernel<<<1,1>>>();
    printf( "Hello, World!\n" );
    return 0;
}
```

- Prazna funkcija nazvana kernel(), sa kvantifikatorom `__global__`
- Poziv ove funkcije, dekorisan sa `<<<1,1>>>`

Say Hello to CUDA (3)

Kernel funkcija sa parametrima

Alokacija

Poziv kernela

Pribavljanje i štampanje rezultata

Dealokacija

```
#include <iostream>
#include "book.h"

__global__ void add(int a, int b, int* c)
{
    *c = a + b;
}

int main( void )
{
    int c;
    int* dev_c;

    HANDLE_ERROR(cudaMalloc((void**)& dev_c, sizeof(int)));

    add<<<1,1>>>(2, 7, dev_c);

    HANDLE_ERROR(cudaMemcpy(&c, dev_c, sizeof(int),
                             cudaMemcpyDeviceToHost));

    printf("2 + 7 = %d\n", c);
    cudaFree( dev_c );
    return 0;
}
```

Say Hello to CUDA (4)

Kernel funkcija sa parametrima

Alokacija

Poziv kernela

Pribavljanje i štampanje rezultata

Dealokacija

```
#include <iostream>
#include "book.h"

__global__ void add(int a, int b, int *c)
{
    *c = a + b;
}

int main( void )
{
    int c;
    int* dev_c;

    HANDLE_ERROR(cudaMalloc((void**)& dev_c, sizeof(int)));

    add<<<1,1>>>(2, 7, dev_c);

    HANDLE_ERROR(cudaMemcpy(&c, dev_c, sizeof(int),
                             cudaMemcpyDeviceToHost));

    printf("2 + 7 = %d\n", c);
    cudaFree( dev_c );
    return 0;
}
```

Say Hello to CUDA (4)

Kernel funkcija sa parametrima

Alokacija

Poziv kernela

Pribavljanje i štampanje rezultata

Dealokacija

```
#include <iostream>
#include "book.h"

__global__ void add(int a, int b, int *c)
{
    *c = a + b;
}

int main( void )
{
    int c;
    int* dev_c;

    HANDLE_ERROR(cudaMalloc((void**)& dev_c, sizeof(int)));

    add<<<1,1>>>(2, 7, dev_c);

    HANDLE_ERROR(cudaMemcpy(&c, dev_c, sizeof(int),
                             cudaMemcpyDeviceToHost));

    printf("2 + 7 = %d\n", c);
    cudaFree( dev_c );
    return 0;
}
```

Sabiranje vektora – Tradicionalni C kod

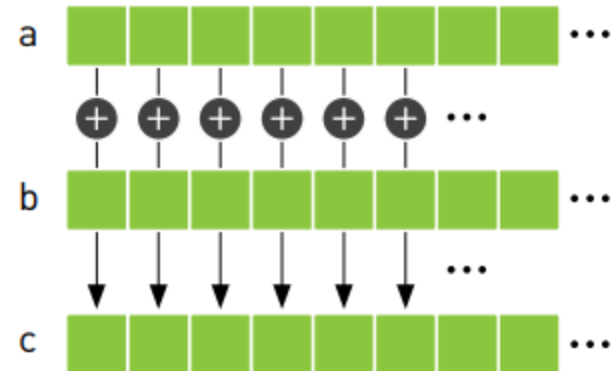
```
#include "book.h"
#define N 10

void add(int* a, int* b, int* c);

int main( void )
{
    int a[N], b[N], c[N];

    // popunjavanje nizova
    for (int i=0; i<N; i++)
    {
        a[i] = -i;
        b[i] = i * i;
    }
    add( a, b, c );
    // štampanje rezultata
    for (int i=0; i<N; i++)
    {
        printf( "%d + %d = %d\n", a[i], b[i], c[i] );
    }

    return 0;
}
```



Sabiranje vektora – Tradicionalni C kod

```
#include "book.h"
```

```
#define N 10
```

```
void add(int* a, int* b, int* c)
```

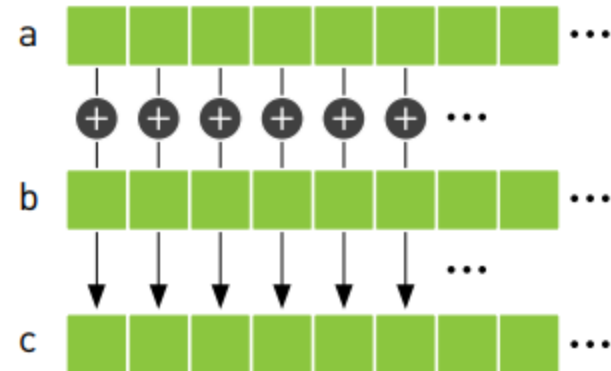
```
{
    for (i=0; i < N; i++)
    {
        c[i] = a[i] + b[i];
    }
}
```

// šta ako na raspolaganju imamo više CPUova?

```
void add2(int* a, int* b, int* c)
```

```
{
    int tid = 0; // CPU 0

    while (tid < N)
    {
        c[tid] = a[tid] + b[tid];
        tid += BROJ_CPUova; // ako imamo samo jedan CPU - ide +1
    }
}
```



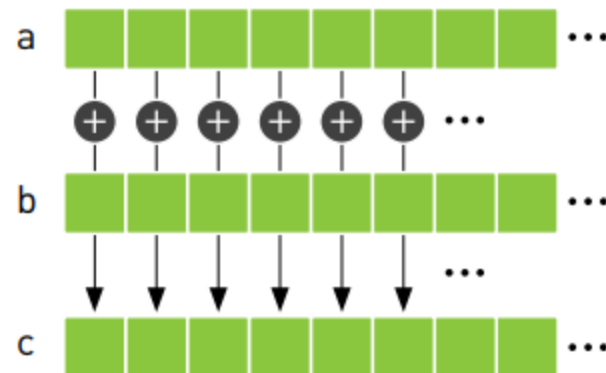
Sabiranje vektora – Tradicionalni C kod

CPU CORE 1

```
void add( int *a, int *b, int *c )
{
    int tid = 0;
    while (tid < N) {
        c[tid] = a[tid] + b[tid];
        tid += 2;
    }
}
```

CPU CORE 2

```
void add( int *a, int *b, int *c )
{
    int tid = 1;
    while (tid < N) {
        c[tid] = a[tid] + b[tid];
        tid += 2;
    }
}
```



Sabiranje vektora – GPU kod (1)

```
#include "book.h"
#define N 10
int main( void )
{
    int a[N];
    int b[N];
    int c[N];
    int* dev_a;
    int* dev_b;
    int* dev_c;

    // alokacija memorije na GPU
    HANDLE_ERROR(cudaMalloc((void**)& dev_a, N * sizeof(int)));
    HANDLE_ERROR(cudaMalloc((void**)& dev_b, N * sizeof(int)));
    HANDLE_ERROR(cudaMalloc((void**)& dev_c, N * sizeof(int)));

    // inicijalizacija nizova a i b, na CPU
    for (int i=0; i<N; i++)
    {
        a[i] = -i;
        b[i] = i * i;
    }
    ...
}
```

Sabiranje vektora – GPU kod (2)

```
...  
// kopiranje nizova a i b na GPU  
HANDLE_ERROR(cudaMemcpy(dev_a, a, N * sizeof(int), cudaMemcpyHostToDevice));  
HANDLE_ERROR(cudaMemcpy(dev_b, b, N * sizeof(int), cudaMemcpyHostToDevice));  
  
add<<<N,1>>>(dev_a, dev_b, dev_c);  
  
// kopiranje niza c sa GPU na CPU  
HANDLE_ERROR(cudaMemcpy(c, dev_c, N * sizeof(int), cudaMemcpyDeviceToHost));  
  
// prikaz rezultata  
for (int i=0; i<N; i++)  
{  
    printf( "%d + %d = %d\n", a[i], b[i], c[i] );  
}  
  
// oslobađanje GPU memorije  
cudaFree( dev_a );  
cudaFree( dev_b );  
cudaFree( dev_c );  
return 0;  
}
```

Sabiranje vektora – GPU kod (3)

```
__global__ void add( int *a, int *b, int *c )  
{  
    // Obrada podataka sa tid indeksom  
    int tid = blockIdx.x;  
  
    if (tid < N)  
        c[tid] = a[tid] + b[tid];  
}
```

- Izvršavanje kernela - broj
- Predefinisane promenljive

Sabiranje vektora – GPU kod (4)

BLOCK 1

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 0;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

BLOCK 2

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 1;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

BLOCK 3

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 2;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

BLOCK 4

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 3;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

Sabiranje vektora – 1 blok, više niti (1)

Izmene:

- Blokovi i niti
 - Ranije smo kreirali N blokova, svaki sa 1 niti
 - `add<<<N,1>>>( dev _ a, dev _ b, dev _ c );`
 - Sad hoćemo 1 blok, svaki sa po N niti
 - `add<<<1,N>>>( dev _ a, dev _ b, dev _ c );`
- Indeksiranje podataka
 - Koristili smo *blockIdx*
 - `int tid = blockIdx.x;`
 - Sada je vrednost *blockIdx* ista za svaku nit, pa koristimo *threadIdx*
 - `int tid = threadIdx.x;`

Sabiranje vektora – 1 blok, više niti (2)

```
#include "../common/book.h"
#define N 10
int main( void )
{
    int a[N];
    int b[N];
    int c[N];
    int* dev_a;
    int* dev_b;
    int* dev_c;

    // alokacija memorije na GPU
    HANDLE_ERROR(cudaMalloc((void**)& dev_a, N * sizeof(int)));
    HANDLE_ERROR(cudaMalloc((void**)& dev_b, N * sizeof(int)));
    HANDLE_ERROR(cudaMalloc((void**)& dev_c, N * sizeof(int)));

    // inicijalizacija nizova a i b, na CPU
    for (int i=0; i<N; i++)
    {
        a[i] = -i;
        b[i] = i * i;
    }
    ...
}
```

Sabiranje vektora – 1 blok, više niti (3)

```

...
// kopiranje nizova a i b na GPU
HANDLE_ERROR(cudaMemcpy(dev_a, a, N * sizeof(int), cudaMemcpyHostToDevice));
HANDLE_ERROR(cudaMemcpy(dev_b, b, N * sizeof(int), cudaMemcpyHostToDevice));

add<<<1,N>>>(dev_a, dev_b, dev_c);

// kopiranje niza c sa GPU na CPU
HANDLE_ERROR(cudaMemcpy(c, dev_c, N * sizeof(int), cudaMemcpyDeviceToHost));

// prikaz rezultata
for (int i=0; i<N; i++)
{
    printf( "%d + %d = %d\n", a[i], b[i], c[i] );
}
// oslobađanje GPU memorije
cudaFree( dev_a );
cudaFree( dev_b );
cudaFree( dev_c );
return 0;
}

```

Sabiranje vektora – 1 blok, više niti (4)

```
__global__ void add( int *a, int *b, int *c )  
{  
    // Obrada podataka sa tid indeksom  
    int tid = threadIdx.x;  
  
    if (tid < N)  
        c[tid] = a[tid] + b[tid];  
}
```

- Izvršavanje kernela - broj
- Predefinisane promenljive

Sabiranje vektora – nedostaci

- Broj blokova koje je moguće pokrenuti u jednoj dimenziji u pozivu je hardverski ograničen na 65 535.
- Broj niti u bloku je takođe ograničen - 1024
- Kako sabrati vektore koji su mnogo veći?
 - Kombinacija blokova i niti

Sabiranje vektora – više blokova, više niti (4)

Izmene:

- Blokovi i niti
 - Sad imamo više blokova i više niti
 - Idalje nam treba N niti, ali ih je potrebno podeliti u više blokova
 - Proizvoljno, možemo uzeti 128 niti po bloku (bitno da bude manje od maksimalnog broja niti po bloku)
 - `add<<<(N+127)/128,128>>>(dev _ a, dev _ b, dev _ c);`
 - Ovakav poziv pokreće više niti nego što je potrebno
- Indeksiranje podataka
 - Koristili smo *blockIdx*, i *threadIdx*
 - Sada indeksiranje izgleda kao konverzija dvodimenzionalnog indeksa u jednodimenzionalni:
 - `int tid = threadIdx.x + blockIdx.x * blockDim.x;`
 - Varijabla *blockDim* čuva broj niti po svakoj dimenziji u bloku

Sabiranje vektora – nedostaci

- Šta kada je broj potrebnih niti veći od maksimalnog broja niti?
 - $\text{max_broj_niti} = \text{max_broj_blokova} * \text{max_broj_niti_u_bloku}$
- Rad sa toliko velikim nizovima nije neuobičajen: današnje kartice imaju dovoljno memorije
 - GeForce GTX 1080 ima 8GB
- Neophodna je izmena kernela

Sabiranje vektora – veliki vektori

```
__global__ void add(int* a, int* b, int* c)
{
    int tid = threadIdx.x +
               blockIdx.x * blockDim.x;

    while (tid < N)
    {
        c[tid] = a[tid] + b[tid];
        tid += blockDim.x * gridDim.x;
    }
}
```

Sličnosti sa CPU implementacijom?

`blockDim.x * gridDim.x` – broj niti

Sabiranje vektora – veliki vektori

- Da bi se izbeglo pokretanje više blokova nego što je neophodno, potrebno je razumno ograničiti broj blokova

```
add<<<128,128>>>( dev _ a, dev _ b, dev _ c );
```

- Koji je sada limit za broj elemenata u vektoru?

Pribavljanje informacija o uređaju

Kako odabrati broj blokova i niti?

- Da bi razvili što optimalnije aplikacije, neophodno je da znamo karakteristike uređaja na kom se aplikacija izvršava
- U slučajevima kada postoji više uređaja, potreban je način za identifikaciju i selekciju

```
int count;
cudaGetDeviceCount(&count));
```

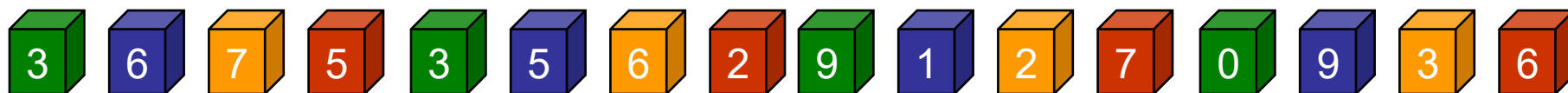
```
struct cudaDeviceProp {
    char name[256];
    size_t totalGlobalMem;
    size_t sharedMemPerBlock;
    int regsPerBlock;
    int warpSize;
    size_t memPitch;
    int maxThreadsPerBlock;
    int maxThreadsDim[3];
    int maxGridSize[3];
    size_t totalConstMem;
    int major;
    int minor;
    int clockRate;
    size_t textureAlignment;
    int deviceOverlap;
    int multiProcessorCount;
    int kernelExecTimeoutEnabled;
    int integrated;
    int canMapHostMemory;
    int computeMode;
    int concurrentKernels;
    int ECCEnabled;
    int pciBusID;
    int pciDeviceID;
    int tccDriver;
}
```

Izbor uređaja

```
int main( void )
{
    cudaDeviceProp prop;
    int dev;
    HANDLE_ERROR(cudaGetDevice(&dev));
    printf("ID of current CUDA device: %d\n", dev);
    memset( &prop, 0, sizeof(cudaDeviceProp));
    prop.major = 1;
    prop.minor = 3;
    HANDLE_ERROR( cudaChooseDevice(&dev, &prop));
    printf("ID of CUDA device closest to revision 1.3:
                                                %d\n", dev);
    HANDLE_ERROR(cudaSetDevice(dev));
}
```

Primer: Pretraživanje niza cifara

- Koliko puta se javlja “6”?
- Niz od 16 elemenata, svaka nit ispituje 4 elementa, jedan blok u gridu
- Ključno:
 - organizacija koda
 - *global*, *host* i *device* funkcije



threadIdx.x = 0 ispituje in_array elemente 0, 4, 8, 12
threadIdx.x = 1 ispituje in_array elemente 1, 5, 9, 13
threadIdx.x = 2 ispituje in_array elemente 2, 6, 10, 14
threadIdx.x = 3 ispituje in_array elemente 3, 7, 11, 15



Ciklična
distribucija
podataka

CUDA Pseudo-kod

GLAVNI PROGRAM:

- Inicijalizacija
 - Alokacija memorije na hostu za ulaze i izlaze podatke
 - Dodela vrednosti ulaznom nizu
- Poziv host funkcije
- Izračunavanje konačnog rezultata na osnovu rezultata pojedinačnih niti
- Prikaz rezultata

GLOBAL FUNKCIJA:

- Nit vrši obilazak svog podskupa niza elemenata
- Poziv device funkcije za poređenje sa "6"
- Izračunavanje lokalnih rezultata

HOST FUNKCIJA:

- Alociranje memorije na uređaju za kopiranje ulaza i izlaza
- Kopiranje ulaza na uređaj
- Postavljanje grida/bloka
- Poziv global funkcije
- Sinhronizacija posle kompletiranja
- Kopiranje izlaza uređaja na host

DEVICE FUNKCIJA:

- Upoređivanje tekućeg elementa sa "6"
- Vрати 1 ako su isti, u suprotnom 0

Glavni program: Preliminarni

GLAVNI PROGRAM:

➤ Inicijalizacija

- Alokacija memorije na hostu za ulazne i izlazne podatke
- Dodela vrednosti ulaznom nizu

➤ Poziv host funkcije

➤ Izračunavanje konačnog rezultata na osnovu rezultata pojedinačnih niti

➤ Prikaz rezultata

```
#include <stdio.h>
#define SIZE 16
#define BLOCKSIZE 4
```

```
int main(int argc, char **argv)
{
    int *in_array, *out_array;
    ...
}
```

Glavni program: Poziv globalne funkcije

GLAVNI PROGRAM:

- Inicijalizacija (preskočeno)
 - Alokacija memorije na hostu za ulazne i izlazne podatke
 - Dodela vrednosti ulaznom nizu
- Poziv host funkcije
- Izračunavanje konačnog rezultata na osnovu rezultata pojedinačnih niti
- Prikaz rezultata

```
#include <stdio.h>
#define SIZE 16
#define BLOCKSIZE 4
__host__ void outer_compute(
    int* in_arr, int* out_arr);

int main(int argc, char **argv)
{
    int *in_array, *out_array;

    /* inicijalizacija */
    ...
    outer_compute(in_array,
out_array);
    ...
}
```

Glavni program: Izračunavanje i prikaz rezultata

GLAVNI PROGRAM:

➤ Inicijalizacija (preskočeno)

- Alokacija memorije na hostu za ulazne i izlazne podatke
- Dodela vrednosti ulaznom nizu

➤ Poziv host funkcije

➤ Izračunavanje konačnog rezultata na osnovu rezultata pojedinačnih niti

➤ Prikaz rezultata

```
#include <stdio.h>
#define SIZE 16
#define BLOCKSIZE 4
__host__ void outer_compute(
    int* in_arr, int* out_arr);
int main(int argc, char **argv)
{
    int *in_array, *out_array;
    int sum = 0;
    /* inicijalizacija */ ...
    outer_compute(in_array, out_array);
    for (int i=0; i<BLOCKSIZE; i++)
    {
        sum+=out_array[i];
    }
    printf ("Result = %d\n",sum);
}
```

Host funkcija: Inicijalizacija

HOST FUNKCIJA:

- Alociranje memorije na uređaju za kopiranje ulaza i izlaza
- Kopiranje ulaza na uređaj
- Postavljanje grida/bloka
- Poziv global funkcije
- Sinhronizacija posle kompletiranja
- Kopiranje izlaza uređaja na host

```
__host__ void outer_compute(  
    int* h_in_array,  
    int* h_out_array)  
{  
    ...  
}
```

Host funkcija: Inicijalizacija

HOST FUNKCIJA:

- Alociranje memorije na uređaju za kopiranje ulaza i izlaza
- Kopiranje ulaza na uređaj
- Postavljanje grida/bloka
- Poziv global funkcije
- Sinhronizacija posle kompletiranja
- Kopiranje izlaza uređaja na host

```
__host__ void outer_compute(  
    int* h_in_array,  
    int* h_out_array)  
{  
    int *d_in_array, *d_out_array;  
  
    cudaMalloc((void **)& d_in_array,  
        SIZE* sizeof(int));  
  
    cudaMalloc((void **)& d_out_array,  
        BLOCKSIZE* sizeof(int));  
  
    ...  
}
```

Host funkcija: Kopiranje podataka

HOST FUNKCIJA:

- Alociranje memorije na uređaju za kopiranje ulaza i izlaza
- Kopiranje ulaza na uređaj
- Postavljanje grida/bloka
- Poziv global funkcije
- Sinhronizacija posle kompletiranja
- Kopiranje izlaza uređaja na host

```
__host__ void outer_compute(  
    int* h_in_array, int* h_out_array)  
{  
    int *d_in_array, *d_out_array;  
  
    cudaMalloc((void **)& d_in_array,  
        SIZE* sizeof(int));  
    cudaMalloc((void **)& d_out_array,  
        BLOCKSIZE* sizeof(int));  
    cudaMemcpy(d_in_array, h_in_array,  
        SIZE*sizeof(int),  
        cudaMemcpyHostToDevice);  
    ... do computation ...  
    cudaMemcpy(h_out_array,  
        d_out_array,  
        BLOCKSIZE*sizeof(int),  
        cudaMemcpyDeviceToHost);  
}
```

Host funkcija: Podešavanja i poziv globalne funkcije

HOST FUNKCIJA:

- Alociranje memorije na uređaju za kopiranje ulaza i izlaza
- Kopiranje ulaza na uređaj
- Postavljanje grida/bloka
- Poziv *global* funkcije
- Sinhronizacija posle kompletiranja
- Kopiranje izlaza uređaja na host

```
__host__ void
outer_compute( int* h_in_array,
               int* h_out_array)
{
    int *d_in_array, *d_out_array;
    cudaMalloc((void **)& d_in_array,
               SIZE* sizeof(int));
    cudaMalloc((void **)& d_out_array,
               BLOCKSIZE* sizeof(int));
    cudaMemcpy(d_in_array, h_in_array,
               SIZE* sizeof(int),
               cudaMemcpyHostToDevice);
    compute<<<(1, BLOCKSIZE)>>>
        (d_in_array, d_out_array);
    cudaThreadSynchronize();
    cudaMemcpy(h_out_array,
               d_out_array,
               BLOCKSIZE* sizeof(int),
               cudaMemcpyDeviceToHost);
}
```

Globalna funkcija

GLOBAL FUNKCIJA:

➤ Nit vrši obilazak svog podskupa niza elemenata

➤ Poziv device funkcije za poređenje sa "6"

➤ Izračunavanje lokalnih rezultata

```
__global__ void compute(int* d_in, int* d_out)
{
    d_out[threadIdx.x] = 0;

    for (int i=0; i<SIZE/BLOCKSIZE; i++)
    {
        int val = d_in[i*BLOCKSIZE + threadIdx.x];
        d_out[threadIdx.x] += compare(val, 6);
    }
}
```

Device funkcija

DEVICE FUNKCIJA:

- Upoređivanje tekućeg elementa sa "6"
- Vrati 1 ako su isti, u suprotnom 0

```
__device__ int compare(int a, int b)
{
    if (a == b)
        return 1;
    return 0;
}
```

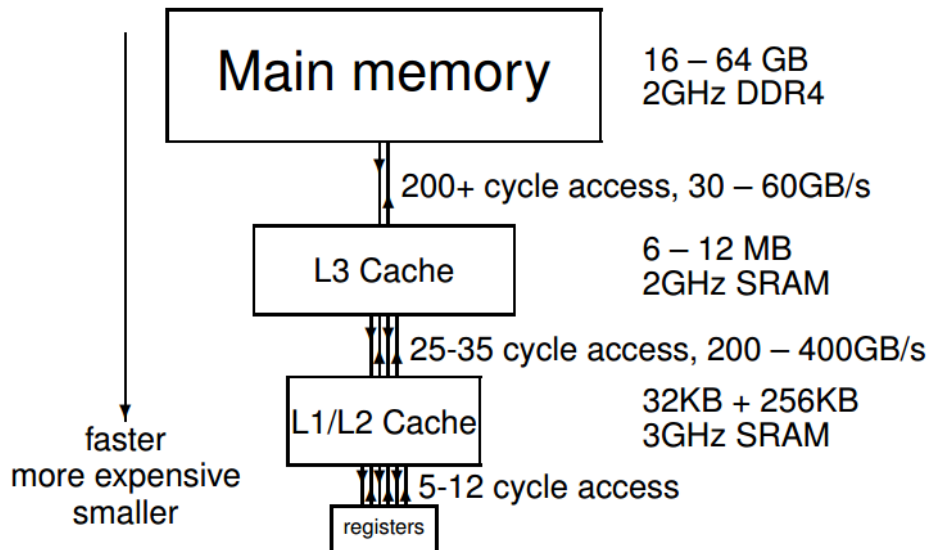
Tipovi promenljivih i memorija

Memorija

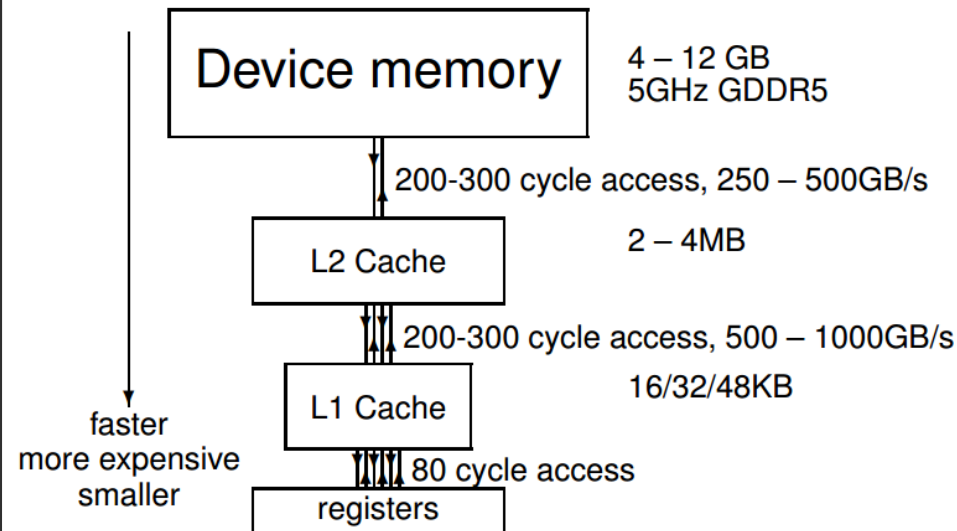
- Jedan od najvećih izazova u modernim računarskim arhitekturama
 - Brza izračunavanja gube poentu ukoliko nije moguće dovoljno brzo prebacivanje podataka
 - Kompleksne aplikacije zahtevaju mnogo memorije
 - Brze memorije su skupe
 - Uvodi se hijerarhijski dizajn memorije
- Brzina izvršavanja se oslanja na lokalnost podataka
 - Temporalna lokalnost
 - Prostorna lokalnost

Memorija

CPU



GPU



Primer 1.

```
__global__ void good_kernel(float *x)
{
    int tid = threadIdx.x + blockDim.x*blockIdx.x;
    x[tid] = threadIdx.x;
}
```

- 32 niti u warpu će adresirati susedne elemente niza x
- Ako su podaci ispravno poravnati (aligned) tako da je x[0] na početku keš linije, onda će i x[0] - x[31] biti u istoj keš liniji (“*coalesced*” transfer)
- Na ovaj način dobija se savršena prostorna lokalost

Primer 2.

```
__global__ void bad_kernel(float *x)
{
    int tid = threadIdx.x + blockDim.x*blockIdx.x;
    x[1000*tid] = threadIdx.x;
}
```

- U ovom slučaju, različite niti iz istog warpa pristupaju ne-susednim elementima niza x („strided“ array access)
- Svaki pristup uključuje različitu keš liniju, što se negativno odražava na performanse

Globalni nizovi

- Čuvaju se u „velikoj“ memoriji device-a
- Alocira ih host kod
- Pokazivače čuva host kod i prosleđuje ih kernelima
- Postoje sve dok ih host kod ne oslobodi
- Pošto se blokovi izvršavaju u proizvoljnom redosledu, ako jedan blok modifikuje elemenat niza, nijedan drugi blok ne treba čitati ili modifikovati taj isti element

Globalne promenljive

- Globalne promenljive se takođe mogu kreirati deklaracijom sa globalnim scope-om unutar fajla sa kernel kodom:

```
__device int reduction_lock=0;
```

```
__global__ void kernel_1(...)  
{  
    ...  
}
```

```
__global__ void kernel_2(...)  
{  
    ...  
}
```

Globalne promenljive

- `__device__` prefiks kaže nvcc-u da se radi o globalnoj promenljivoj na GPU, ne na CPU
- Promenljivu može da čita i modifikuje bilo koji kernel
- Životni vek joj je isti kao životni vek aplikacije
- Moguće je deklarirati i nizove fiksne dužine
- Moguće je čitanje i modifikacija od strane host koda korišćenjem specijalnih rutina `cudaMemcpyToSymbol`, `cudaMemcpyFromSymbol` ili standardne `cudaMemcpy` u kombinaciji sa `cudaGetSymbolAddress`

Konstante

- Jako slične globalnim promenljivama, osim što kerneli ne mogu da ih modifikuju
 - Definišu se globalnim scope-om unutar kernel fajla korišćenjem prefiksa `__constant__`
 - Inicijalizuje ih host kod korišćenjem `cudaMemcpyToSymbol`, `cudaMemcpyFromSymbol` ili `cudaMemcpy` u kombinaciji sa `cudaGetSymbolAddress`
- Dostupno je samo 64KB memorije za konstante, ali svaki SM ima 8-10KB keša
 - Kada sve niti čitaju istu konstantu, brzina čitanja jednaka je brzini čitanja iz registara
 - Ne zauzima registre, pa može biti od koristi za minimizaciju broja potrebnih registara

Konstante

- Vrednost konstante se postavlja u run-time-u.
- Kod često sadrži konstante čija se vrednost zna u vremenu kompajliranja:

```
#define PI 3.1415926f  
a = b / (2.0f * PI);
```

- Ovakve konstante se ugrađuju u executable kod, pa ne zauzimaju registre
- f na kraju se koristi za jednostruku preciznost, jer je u C/C++:

single x double = double

Registri

- U okviru svakog kernela podrazumevano se individualne promenljive smeštaju u registre:

```
__global__ void lap(int I, int J, float *u1, float *u2)
{
    int i = threadIdx.x + blockIdx.x*blockDim.x;
    int j = threadIdx.y + blockIdx.y*blockDim.y;
    int id = i + j*I;

    if (i==0 || i==I-1 || j==0 || j==J-1)
        u2[id] = u1[id];
    else
        u2[id] = 0.25f * (u1[id-1] + u1[id+1] + u1[id-I]
                          + u1[id+I]);
}
```

Registri

- 64K 32-bit registara po SM
- Do 255 registara za svaku nit
- Do 2048 niti (najviše 1024 niti po bloku)
- Max registara za svaku nit => 256 niti
- Max broj niti => 32 registara za svaku nit
- Postoji velika razlika između “fat” i “thin” niti
- Ako je aplikaciji potrebno više registara, koristi se najpre L1 keš, a posle i memorija device-a
- Nije definisano da li konkretna promenljiva postaje niz sa jednim elementom na uređaju ili se vrednosti u registrima “čuvaju” u memoriji device-a, a kasnije se vraćaju
- U svakom slučaju, javlja se latenca usled korišćenja memorije uređaja

Lokalni nizovi

Šta se dešava ako aplikacija koristi nizove sa malim broj elemenata?

```
__global__ void lap(float *u)
{
    float ut[3];
    int tid = threadIdx.x + blockIdx.x*blockDim.x;

    for (int k=0; k<3; k++)
        ut[k] = u[tid+k*gridDim.x*blockDim.x];

    for (int k=0; k<3; k++)
        u[tid+k*gridDim.x*blockDim.x] =
            A[3*k]*ut[0]+A[3*k+1]*ut[1]+A[3*k+2]*ut[2];
}
```

Lokalni nizovi

- U jednostavnim situacijama (koje su jako česte), kompajler ove nizove konvertuje u skalare i smešta ih u registre:

```
__global__ void lap(float* u)
{
    int tid = threadIdx.x + blockIdx.x*blockDim.x;

    float ut0 = u[tid + 0*gridDim.x*blockDim.x];
    float ut1 = u[tid + 1*gridDim.x*blockDim.x];
    float ut2 = u[tid + 2*gridDim.x*blockDim.x];

    u[tid + 0*gridDim.x*blockDim.x] = A[0]*ut0 + A[1]*ut1 + A[2]*ut2;
    u[tid + 1*gridDim.x*blockDim.x] = A[3]*ut0 + A[4]*ut1 + A[5]*ut2;
    u[tid + 2*gridDim.x*blockDim.x] = A[6]*ut0 + A[7]*ut1 + A[8]*ut2;
}
```

Lokalni nizovi

- Kod malo komplikovanijih situacija, kompajler stavlja nizove u memoriju device-a
 - Idalje se radi o lokalnom nizu jer svaka nit poseduje svoju privatnu kopiju
 - Podrazumevano se čuvaju u L1 kešu, i često se dešava da nikad ne budu premešteni u memoriju device-a
 - 48kB L1 keša se slika na 12k 32-bitnih promenljivih
=> 12 za svaku nit ako se koristi 1024 niti

Deljena memorija

- U kernelima, prefiks `__shared__` deklarira podatke kao deljive između svih niti u bloku niti – svaka nit može čitati ili modifikovati vrednost:
`__shared__ int x_dim;`
`__shared__ float x[128];`
- Ovakvi podaci su neophodni za operacije koje zahtevaju komunikaciju između niti
- Korisni za re-use
- Alternativa za lokalne nizove u memoriji device-a

Deljena memorija

- Ako blok niti ima više od jednog warpa, nije definisano kada će koji warp krenuti i završiti
- Zbog toga je skoro uvek neophodna sinhronizacija niti kako bi se obezbedilo korektno korišćenje deljive memorije
- Instrukcija `__syncthreads()` postavlja “barijeru”: Nijedna nit/warp ne može nastaviti izvršenje iza barijere sve dok svi ostali nisu stigli do barijere
- Pored statički alocirane deljive memorije, moguće je i kreiranje dinamičkih nizova u deljivoj memoriji
- Ukupna veličina specificira se opcionim trećim argumentom pri pozivu kernela:

```
kernel<<<blocks, threads, shared_bytes>>>(...)
```

Read-only nizovi

- Pri radu sa konstantama, svaka nit čita istu vrednost
- U drugim slučajevima, imamo nizove čije se vrednosti elemenata ne menjaju, ali različite niti čitaju različite elemente
- Tada je korisno naglasiti kompajleru da se radi o read-only nizu:

```
const __restrict__
```

- Na hardeverskom nivou, koriste se instrukcije koje daju bolje performanse

CUDA kvalifikatori promenljivih

- Automatske promenljive bez kvalifikatora se smeštaju u registre
 - Osim velikih struktura i statičkih nizova koji se smeštaju u lokalnu memoriju
- Pokazivači mogu da pokazuju samo na objekte iz globalne memorije:
 - Alocirane na strani domaćina i prosleđene jezgru
`__global__ void KernelFunc(float* ptr);`
 - Statički deklarisanе objekte na strani uređaja
`float* ptr = &globalVar;`
- Kvalifikator **__device__** je opcion ako su navedeni kvalifikatori **__shared__** ili **__constant__**

		Memorija	Opseg	Životni vek
<code>__device__ __shared__</code>	<code>int SharedVar</code>	Deljena	Blok	Blok
<code>__device__</code>	<code>int GlobalVar</code>	Globalna	Grid	Aplikacija
<code>__device__ __constant__</code>	<code>int ConstantVar</code>	Konstantna	Grid	Aplikacija

Memorija - hijerarhija

- Globalna memorija (GB)
- Deljena memorija (kB)
- Registri
- Lokalna memorija
- Constant memorija
- Texture memorija

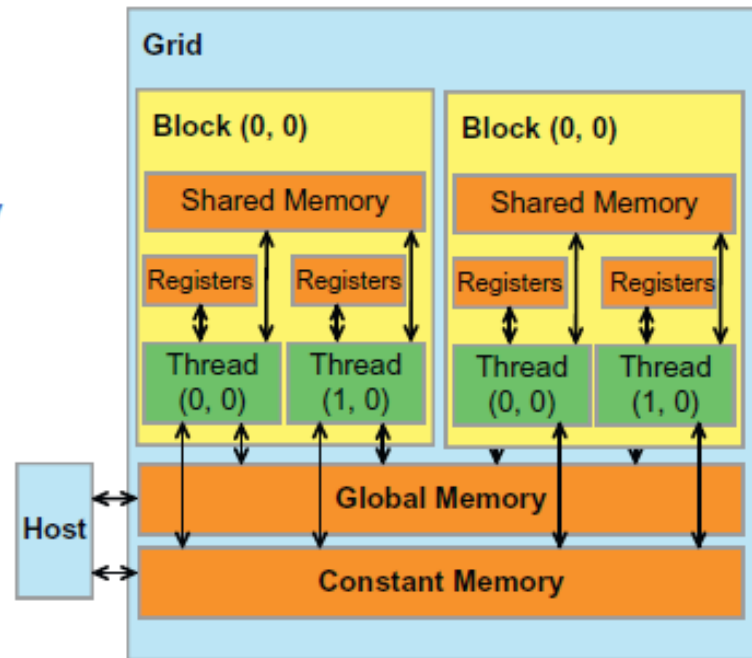
Memorija - hijerarhija

Device code can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

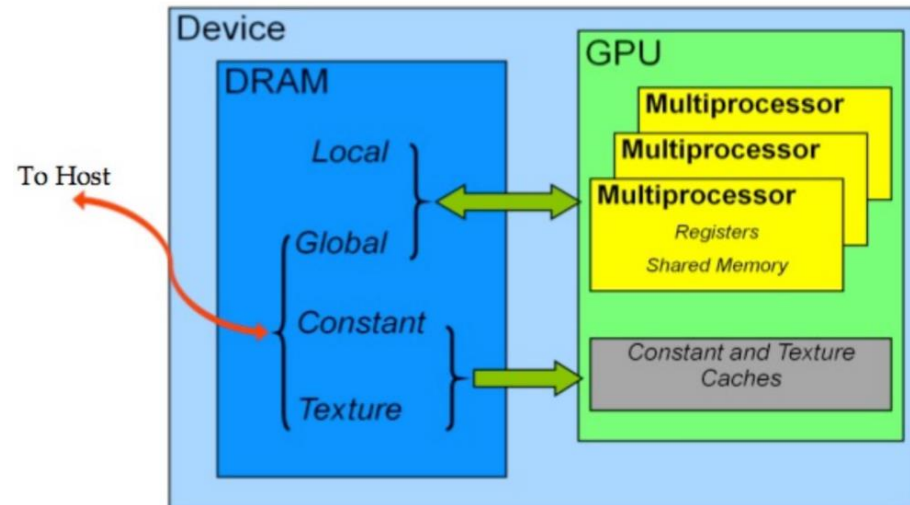
Host code can

- Transfer data to/from per grid **global** and **constant** memories



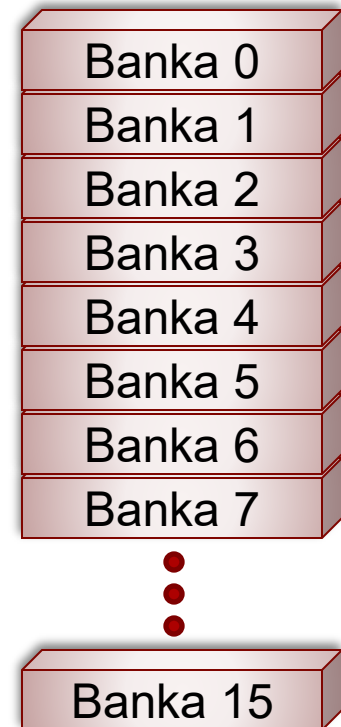
Memorija - hijerarhija

Memory	Location on/off chip	Cached	Access	Scope	Lifetime
Register	On	n/a	R/W	1 thread	Thread
Local	Off	†	R/W	1 thread	Thread
Shared	On	n/a	R/W	All threads in block	Block
Global	Off	†	R/W	All threads + host	Host allocation
Constant	Off	Yes	R	All threads + host	Host allocation
Texture	Off	Yes	R	All threads + host	Host allocation



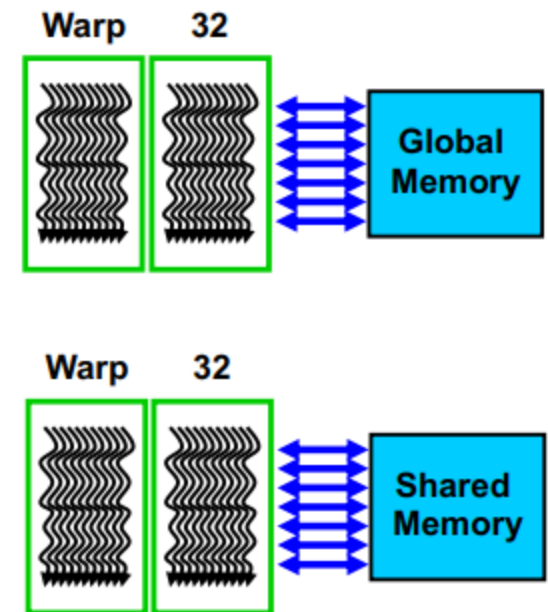
Paralelna memorijska arhitektura (1)

- Kod paralelne mašine, veliki broj niti pristupa memoriji
 - Memorija je preklopljena i podeljena u banke
 - I globalna i deljena memorija
 - Vrlo bitno za postizanje velikog propusnog opsega
- Svaka memorijska banka može da usluži jedan zahtev u jednom ciklusu
 - Celokupna memorija može simultano da usluži onoliko pristupa koliko ima memorijskih banki
- Više simultanih pristupa istoj banki dovodi do konflikta
 - Konfliktne pristupi se serijalizuju



Paralelna memorijska arhitektura (2)

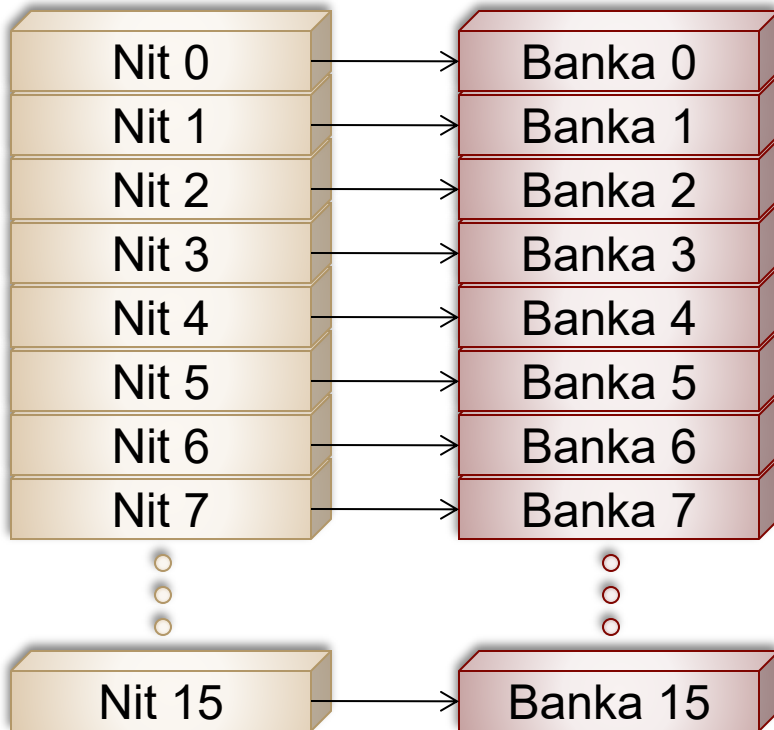
- Na novijim procesorima, memorija je podeljena u 32 banke
 - Uzastopne 32-bitne reči se dodeljuju uzastopnim memorijskim bankama
- Pristup memoriji na CUDA se kombinuje u transakcije
 - Najbolje performanse se dobijaju kada sve niti unutar warp-a pristupaju uzastopnim memorijskim lokacijama
 - Tada nema konflikata
 - Konflikti su mogući jedino unutar warp-a



Primeri pristupa memoriji (1)

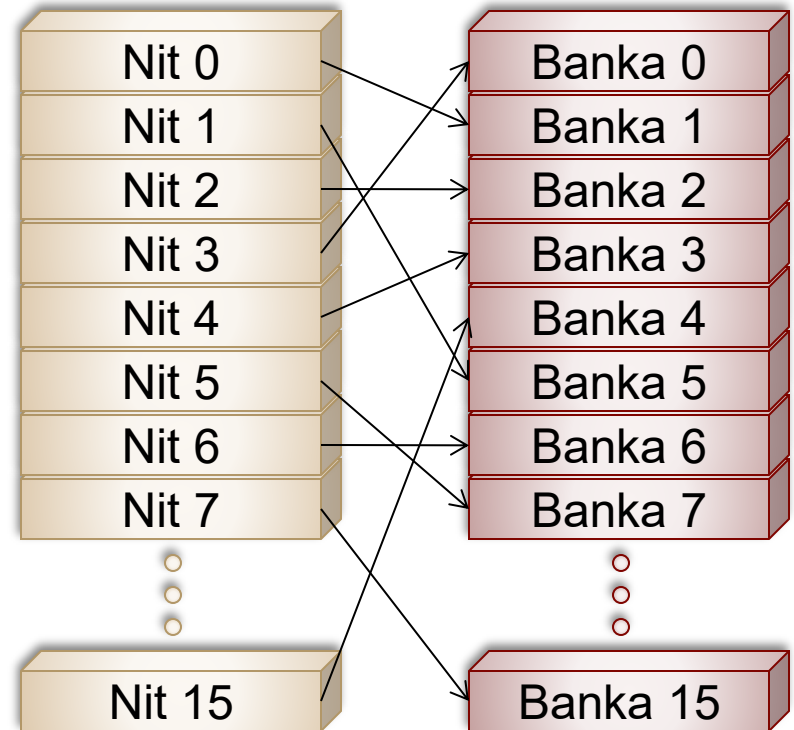
Nema konflikata

Linearno adresiranje, stride = 1



Nema konflikata

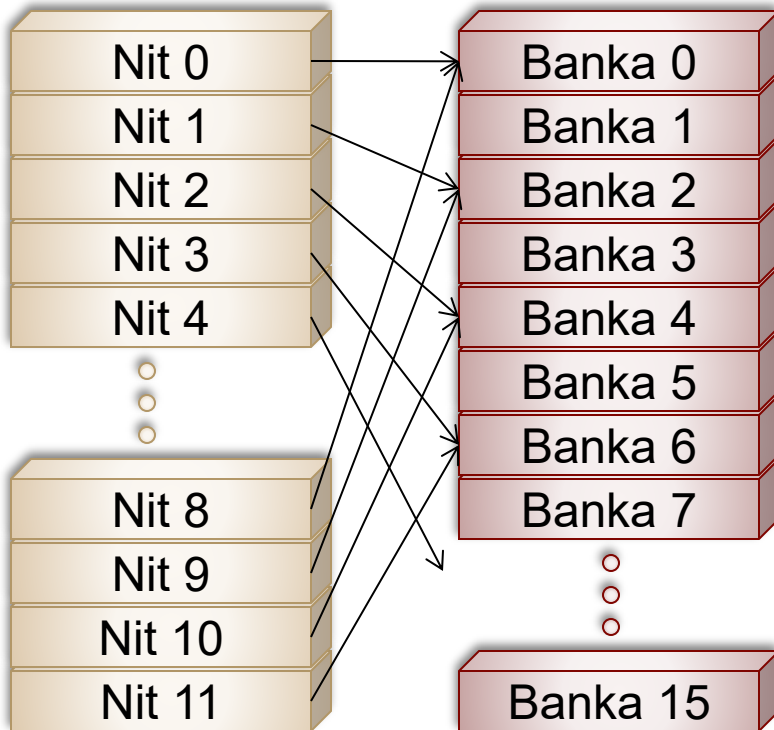
Slučajan pristup memoriji



Primeri pristupa memoriji (2)

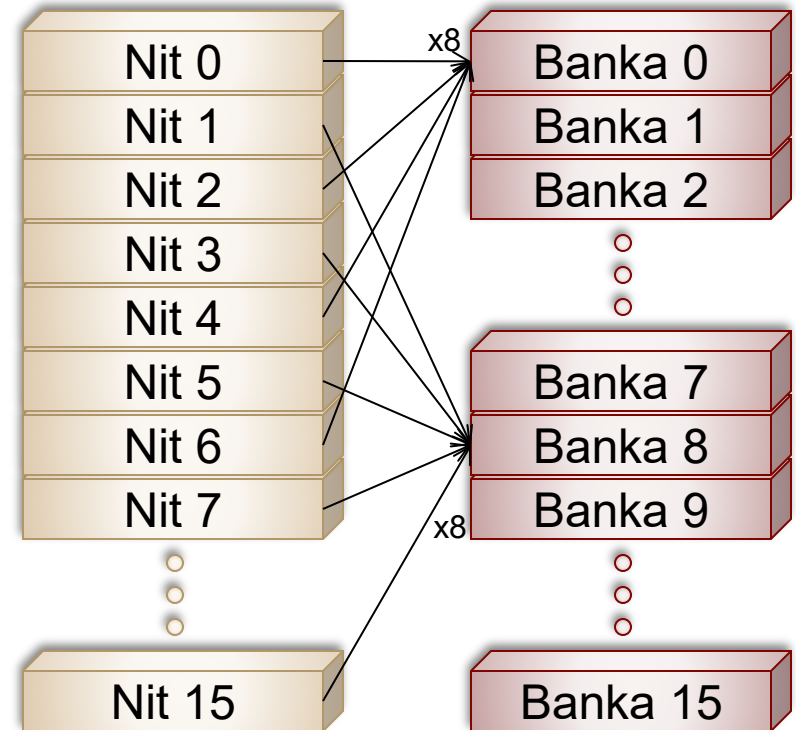
Dvostruki konflikt

Linearno adresiranje, stride = 2



8-struki konflikt

Linearno adresiranje, stride = 8



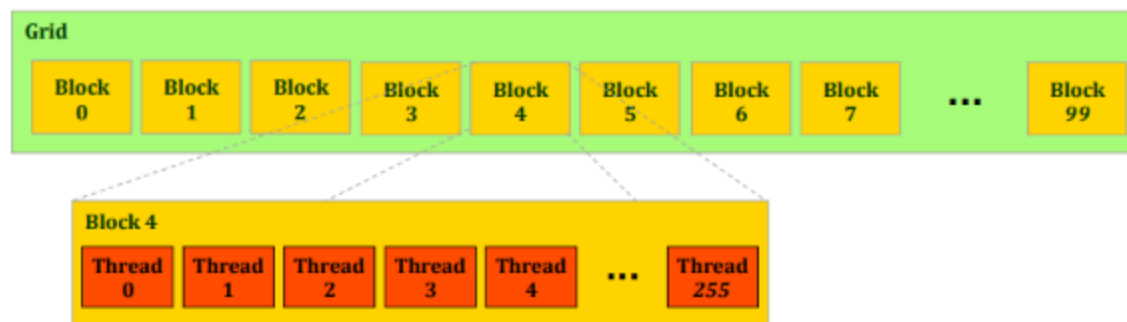
Adresiranje niti – 1D

```

__global__ void kernel1(float *idata, float *odata)
{
    int i;
    i = blockIdx.x * blockDim.x + threadIdx.x;
    odata[i] = func(idata[i]);
}

...
dim3 gridDef1(100,1,1);
dim3 blockDef1(256,1,1);
kernel1<<<gridDef1, blockDef1>>>(paramList);

```



Adresiranje niti – 2D

```

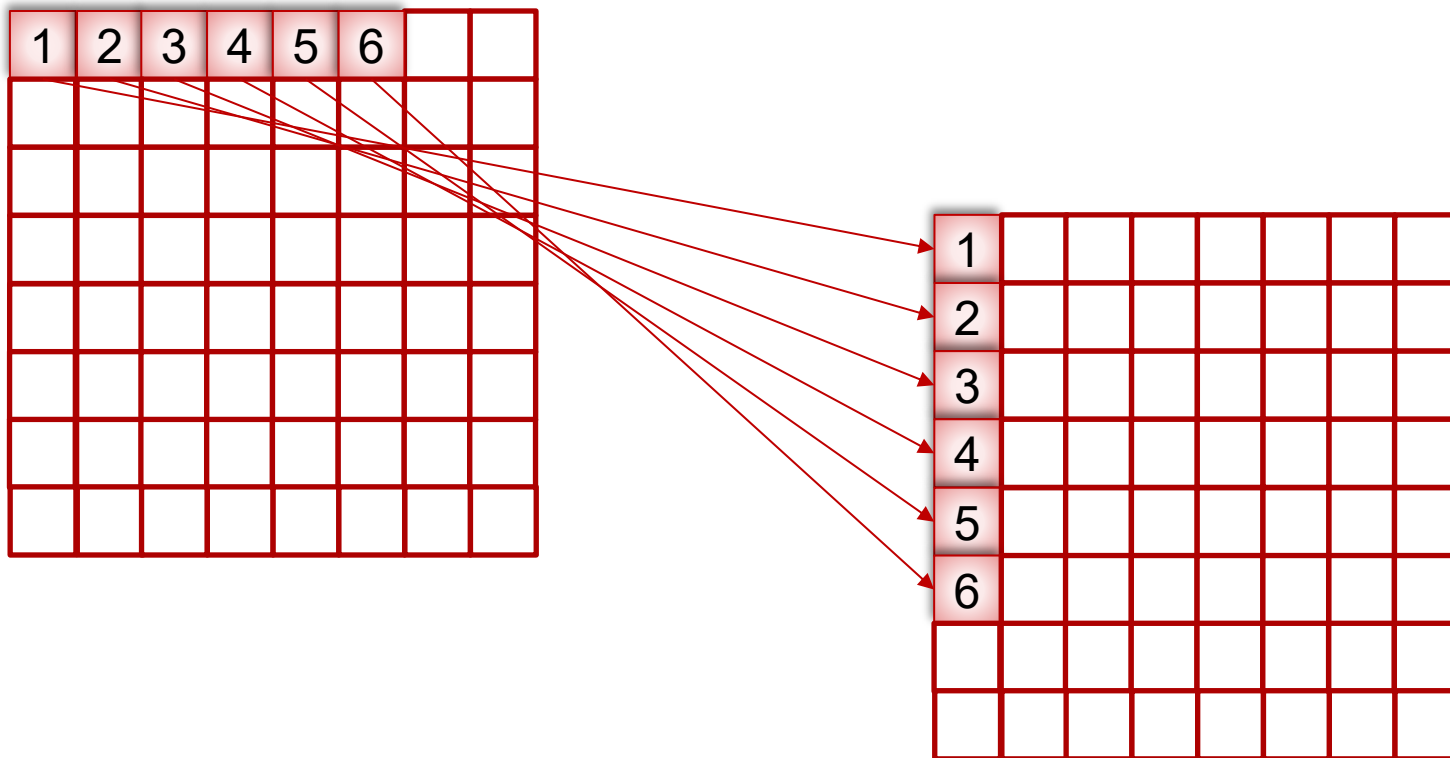
__global__ void kernel2(float *idata, float *odata, int pitch)
{
    int x, y, i;
    x = blockIdx.x * blockDim.x + threadIdx.x;
    y = blockIdx.y * blockDim.y + threadIdx.y;
    i = y * pitch + x;
    odata[i] = func(idata[i]);
}
...
dim3 gridDef2(3,2,1);
dim3 blockDef2(5,3,1);
kernel2<<<gridDef2, blockDef2>>>(paramList);

```



Transponovanje matrica

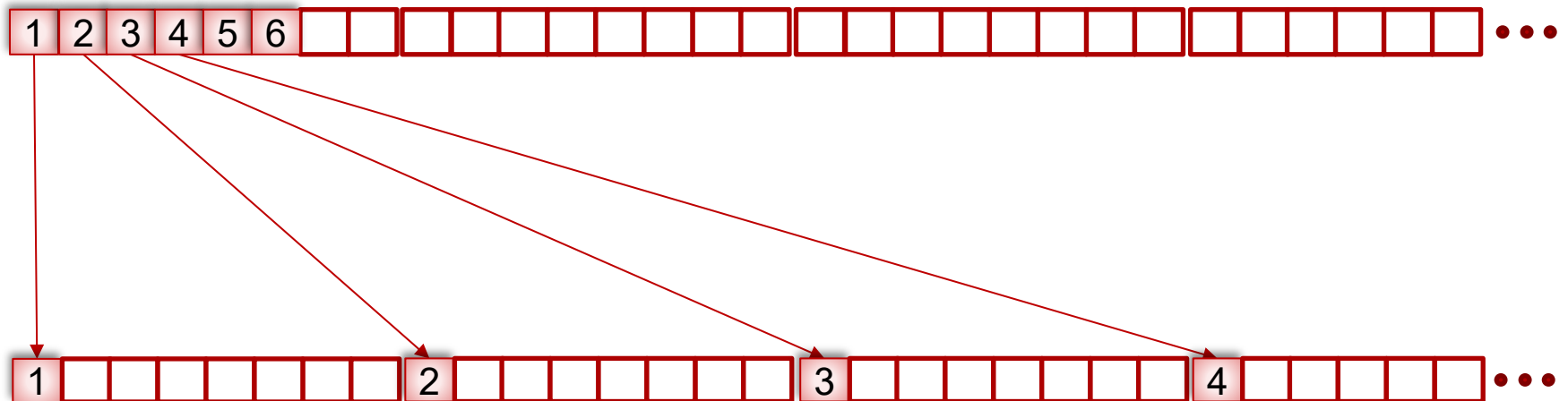
- Vrste ulazne matrice postaju kolone izlazne matrice



Jednostavno rešenje

```
__global__ void transpose_naive(  
    float *odata, float *idata, int width, int height)  
{  
    int xIndex, yIndex, index_in, index_out;  
  
    xIndex = blockDim.x * blockIdx.x + threadIdx.x;  
    yIndex = blockDim.y * blockIdx.y + threadIdx.y;  
  
    if (xIndex < width && yIndex < height)  
    {  
        index_in = xIndex + width * yIndex;  
        index_out = yIndex + height * xIndex;  
        odata[index_out] = idata[index_in];  
    }  
}
```

Jednostavno rešenje - problemi



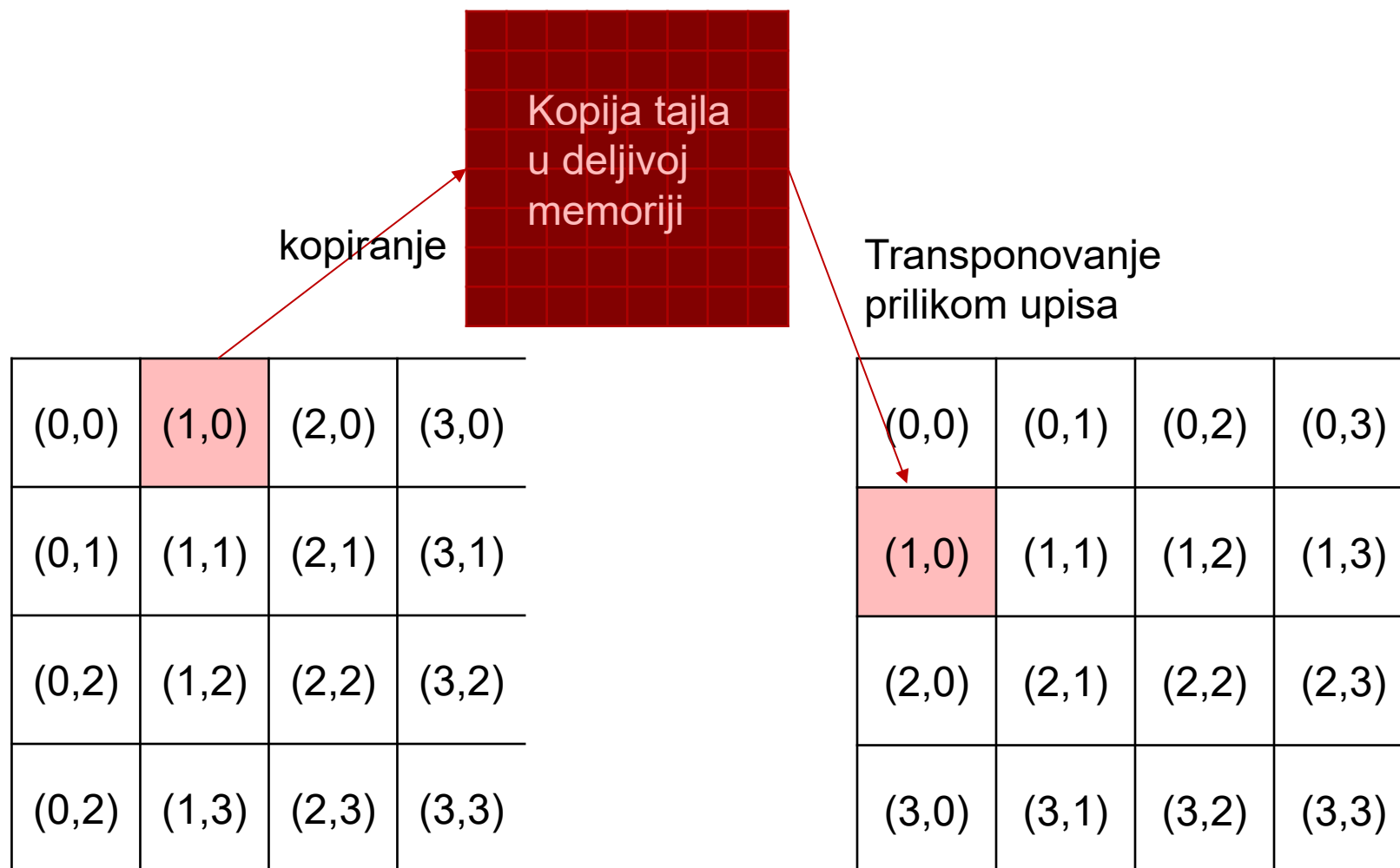
Moguće rešenje

- Navedeni problem može biti adresiran korišćenjem deljive memorije uređaja
- Svaki SM na GPU ima svoju deljivu memoriju
- Niti u okviru jednog bloka mogu da komuniciraju putem deljive memorije
- Svaka nit u okviru bloka ima R/W pristup deljivoj memoriji koja je alocirana bloku
- Niti se mogu sinhronizovati pomoću `__syncthreads()`;

Korišćenje deljive memorije

- Matricu delimo na tajlove veličine 32×32 . Svaki tajl će procesirati poseban blok
- Blok niti će učitati svoj tajl iz ulazne matrice u deljivu memoriju
- Svaki tajl se transpose-uje tako što se elementi upišu na odgovarajuća mesta u izlaznoj matrici
- Sada je i upis u izlaznu matricu coalesced

Optimizovano transponovanje matrica



Optimizovano transponovanje matrica - 1

```
__global__ void transpose(float *odata, float *idata, int width, int height)
{
    __shared__ float block[BLOCK_DIM][BLOCK_DIM];
    unsigned int xIndex, yIndex, index_in, index_out;

    xIndex = blockIdx.x * BLOCK_DIM + threadIdx.x;
    yIndex = blockIdx.y * BLOCK_DIM + threadIdx.y;

    if ((xIndex < width) && (yIndex < height))
    {
        index_in = yIndex * width + xIndex;
        block[threadIdx.y][threadIdx.x] = idata[index_in];
    }

    __syncthreads();
}
```

BLOCK_DIM = 32

Učitavanje tajla u
shared memoriju

Optimizovano transponovanje matrica

Upis transponovanog
tajla u izlaznu matricu

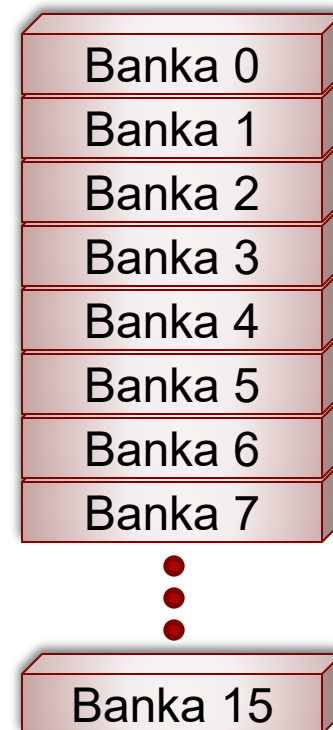
```
xIndex = blockIdx.y * BLOCK_DIM + threadIdx.x;  
yIndex = blockIdx.x * BLOCK_DIM + threadIdx.y;  
  
if ((xIndex < height) && (yIndex < width))  
{  
    index_out = yIndex * height + xIndex;  
    odata[index_out] = block[threadIdx.x][threadIdx.y];  
}  
}
```



Problem?

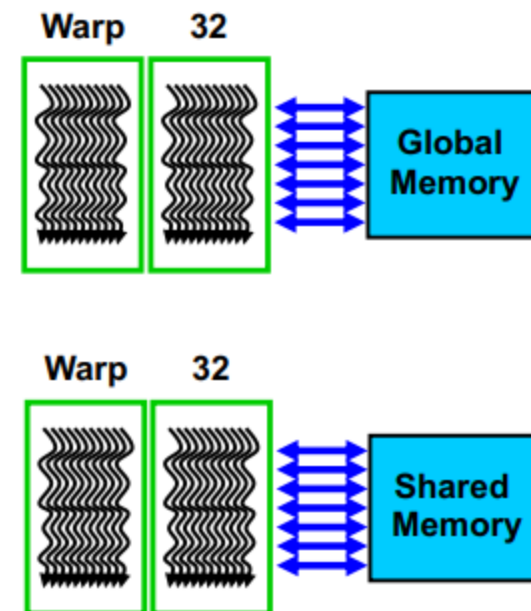
Paralelna memorijska arhitektura (1)

- Kod paralelne mašine, veliki broj niti pristupa memoriji
 - Memorija je preklopljena i podeljena u banke
 - I globalna i deljena memorija
 - Vrlo bitno za postizanje velikog propusnog opsega
- Svaka memorijska banka može da usluži jedan zahtev u jednom ciklusu
 - Celokupna memorija može simultano da usluži onoliko pristupa koliko ima memorijskih banki
- Više simultanih pristupa istoj banki dovodi do konflikta
 - Konfliktne pristupi se serijalizuju



Paralelna memorijska arhitektura (2)

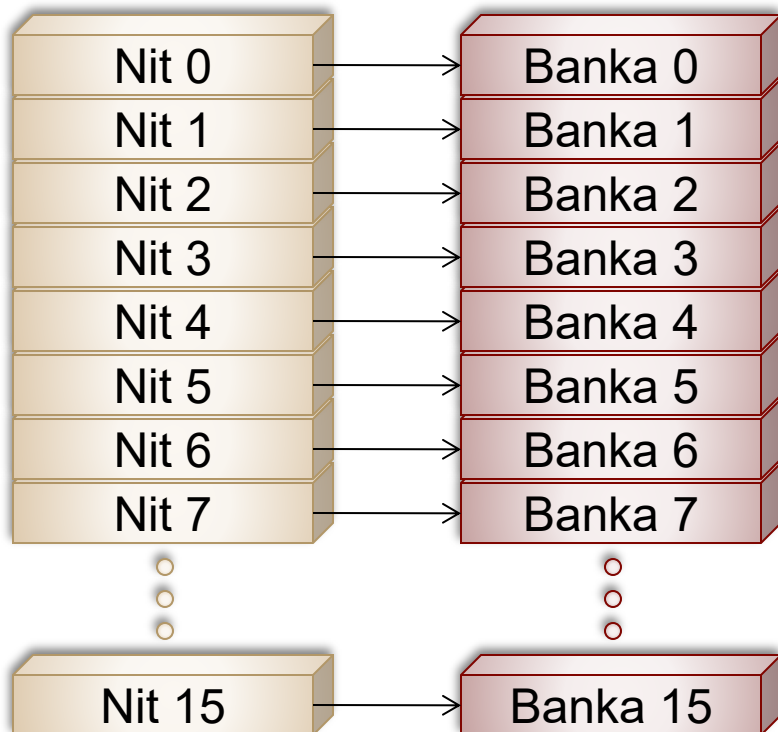
- Na novijim procesorima, memorija je podeljena u 32 banke
 - Uzastopne 32-bitne reči se dodeljuju uzastopnim memorijskim bankama
- Pristup memoriji na CUDA se kombinuje u transakcije
 - Najbolje performanse se dobijaju kada sve niti unutar warp-a pristupaju uzastopnim memorijskim lokacijama
 - Tada nema konflikata
 - Konflikti su mogući jedino unutar warp-a



Primeri pristupa memoriji (1)

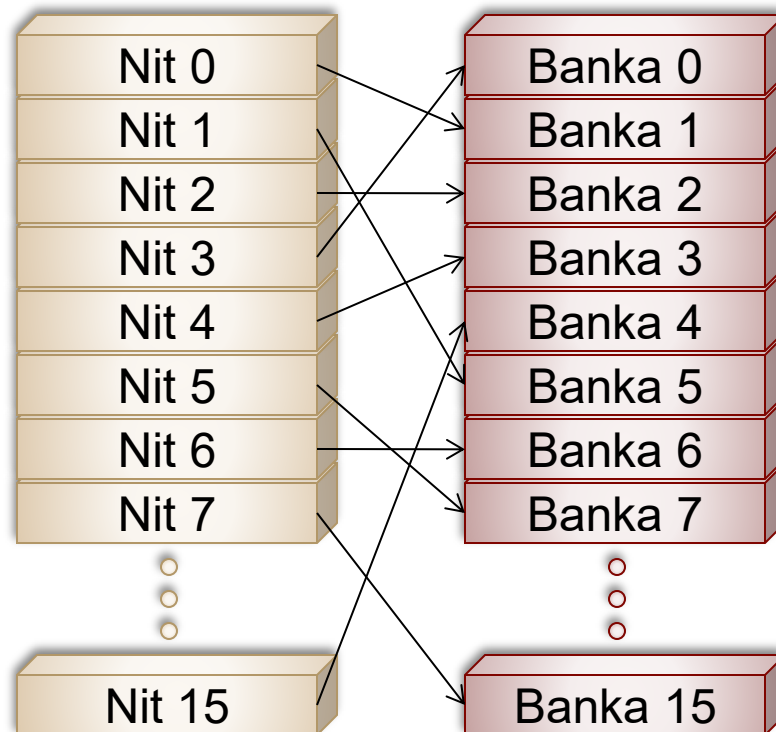
Nema konflikata

Linearno adresiranje, stride = 1



Nema konflikata

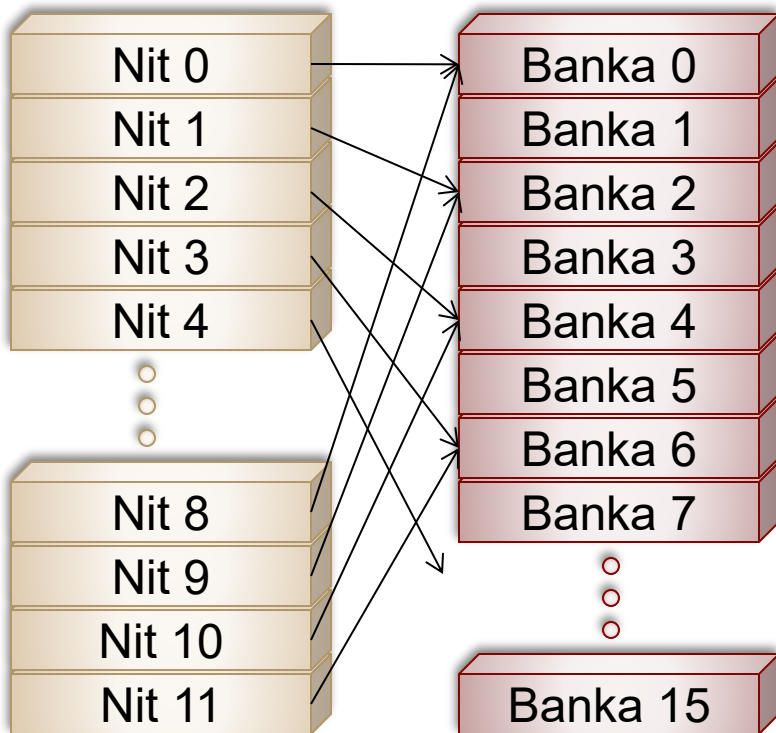
Slučajan pristup memoriji



Primeri pristupa memoriji (2)

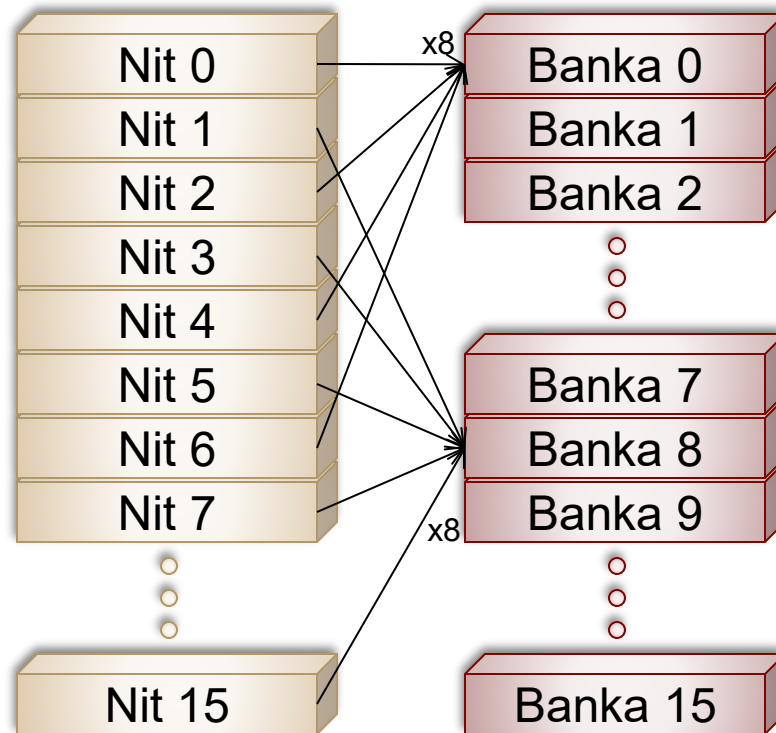
Dvostruki konflikt

Linearno adresiranje, stride = 2



8-struki konflikt

Linearno adresiranje, stride = 8



Transponovanje matrica i konflikti

- Konfliktne pristupi memorijskim bankama se dešavaju kada se deljivoj memori pristupa preko kolona (prilikom upisa tajla)
- Sve niti u okviru jednog warp-a pristupaju adresama koje imaju međusobni ofset jednak BLOCK_DIM elemenata
 - BLOCK_DIM = 32
- Pošto imamo 32 banke, svi pristupi gađaju istu banku!

Rešenje

- Umesto da alociramo $\text{BLOCK_DIM} \times \text{BLOCK_DIM}$ tajl u deljivoj memoriji, alociramo $\text{BLOCK_DIM} \times (\text{BLOCK_DIM} + 1)$ tajl
- Ova izmena je dovoljna da sve konkurentne niti pristupaju različitim bankama deljive memorije
 - Kolone više nisu poravnate na 32
 - Nikakve dodatne izmene u kodu nisu potrebne

Optimizovano transponovanje matrica - 2

```
__global__ void transpose(float *odata, float *idata, int width, int height)
{
    __shared__ float block[BLOCK_DIM][BLOCK_DIM+1];
    unsigned int xIndex, yIndex, index_in, index_out;

    /* read the matrix tile into shared memory */
    xIndex = blockIdx.x * BLOCK_DIM + threadIdx.x;
    yIndex = blockIdx.y * BLOCK_DIM + threadIdx.y;

    if ((xIndex < width) && (yIndex < height))
    {
        index_in = yIndex * width + xIndex;
        block[threadIdx.y][threadIdx.x] = idata[index_in];
    }

    __syncthreads();
}
```

BLOCK_DIM = 32

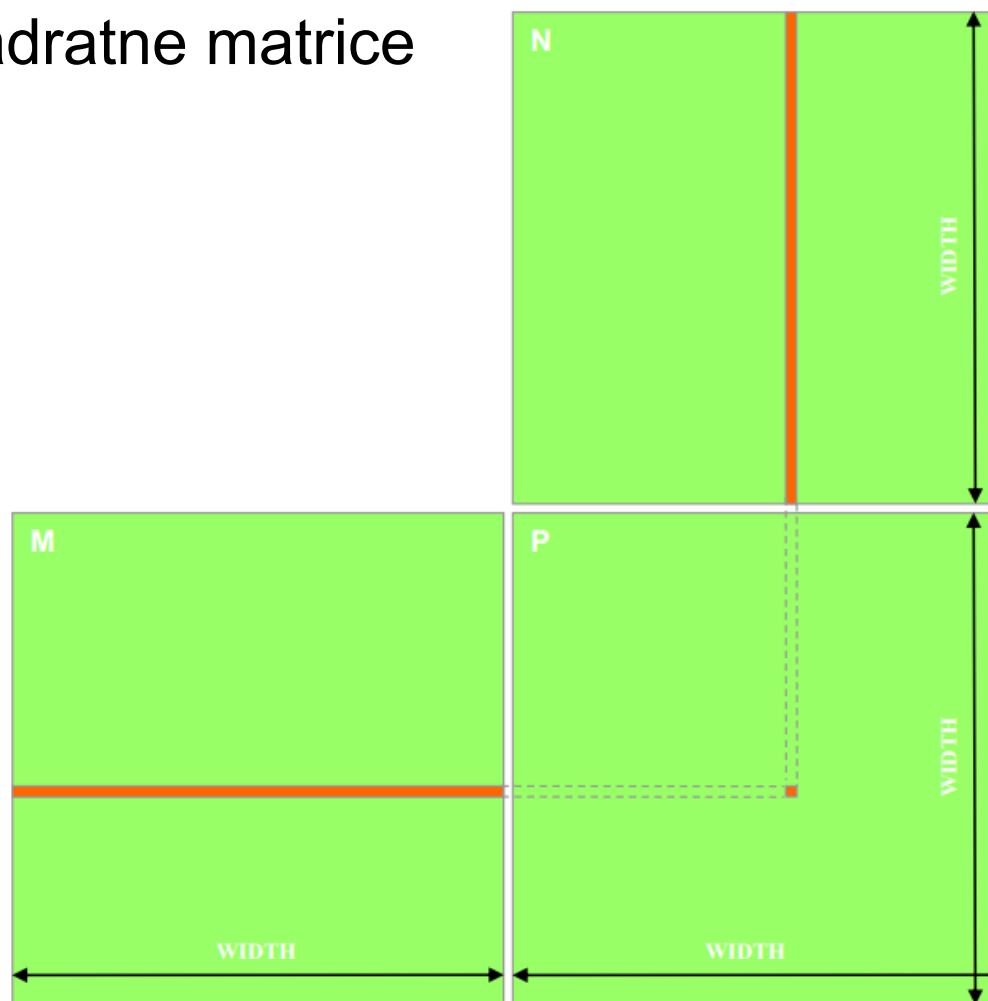
Optimizovano transponovanje matrica

```
/* write the transposed matrix tile to global memory */
xIndex = blockIdx.y * BLOCK_DIM + threadIdx.x;
yIndex = blockIdx.x * BLOCK_DIM + threadIdx.y;

if ((xIndex < height) && (yIndex < width))
{
    index_out = yIndex * height + xIndex;
    odata[index_out] = block[threadIdx.x][threadIdx.y];
}
}
```

Primer: Množenje matrica

- Pomnožiti dve kvadratne matrice



Primer: Množenje matrica (2)

- Tradicionalni sekvencijalni kod:

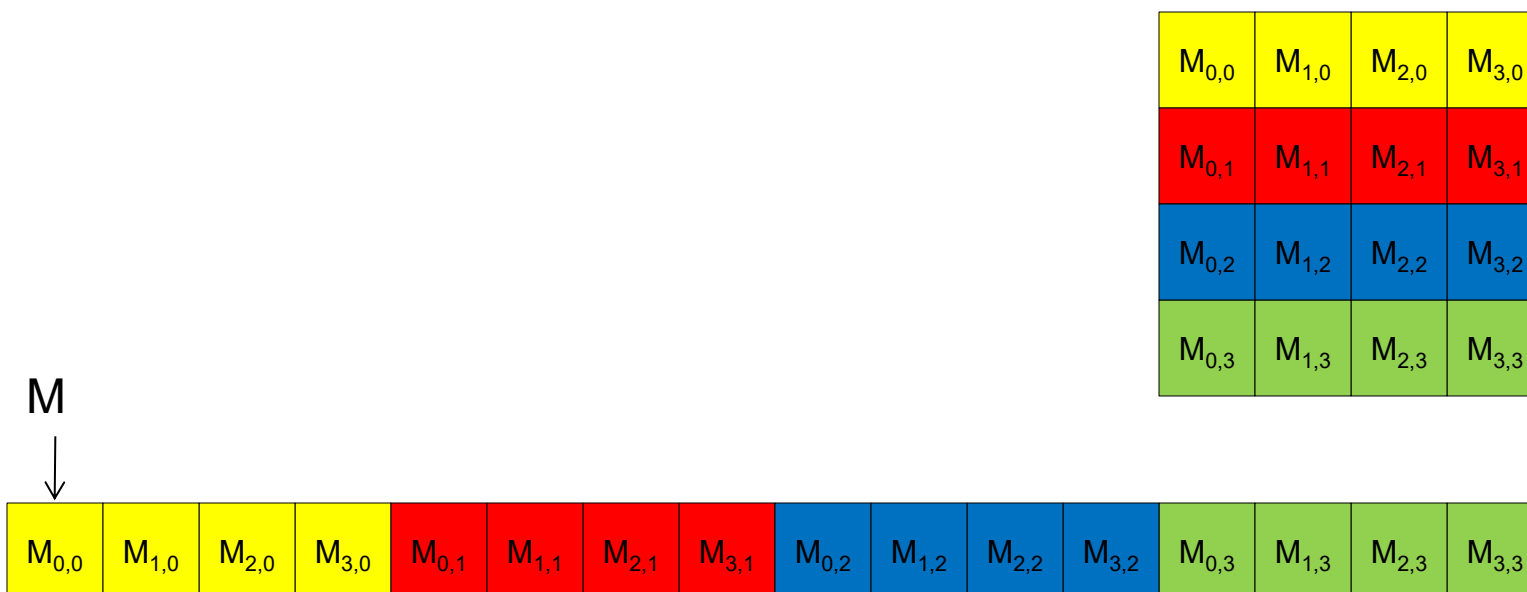
```
void MatrixMulOnHost (float* M, float* N, float* P, int Width)
{
    for (int i = 0; i < Width; ++i)
        for (int j = 0; j < Width; ++j)
        {
            float sum = 0;

            for (int k = 0; k < Width; ++k)
            {
                float a = M[i * width + k];
                float b = N[k * width + j];
                sum += a * b;
            }

            P[i * Width + j] = sum;
        }
}
```

Primer: Množenje matrica (3)

- Matrice se u C-u smeštaju po vrstama
 - Matrica će uređaju biti preneti linearizovana
 - Svaka nit će proračunati adresu elementa kome treba da pristupi



Primer: Množenje matrica (4)

HOST:

```
void MatrixMulOnDevice (float* M, float* N, float* P, int Width)
{
    int size = Width * Width * sizeof(float);
    float* Md;
    float* Nd;
    float* Pd;

    // Allocate and Load M, N to device memory
    cudaMalloc(&Md, size);
    cudaMemcpy(Md, M, size, cudaMemcpyHostToDevice);
    cudaMalloc(&Nd, size);
    cudaMemcpy(Nd, N, size, cudaMemcpyHostToDevice);

    // Allocate P on the device
    cudaMalloc(&Pd, size);

    //////////////////////////////////////
    // Kernel invocation code - to be shown later
    //////////////////////////////////////

    // Read P from the device
    cudaMemcpy(P, Pd, size, cudaMemcpyDeviceToHost);
    // Free device matrices
    cudaFree(Md);
    cudaFree(Nd);
    cudaFree (Pd);
}
```

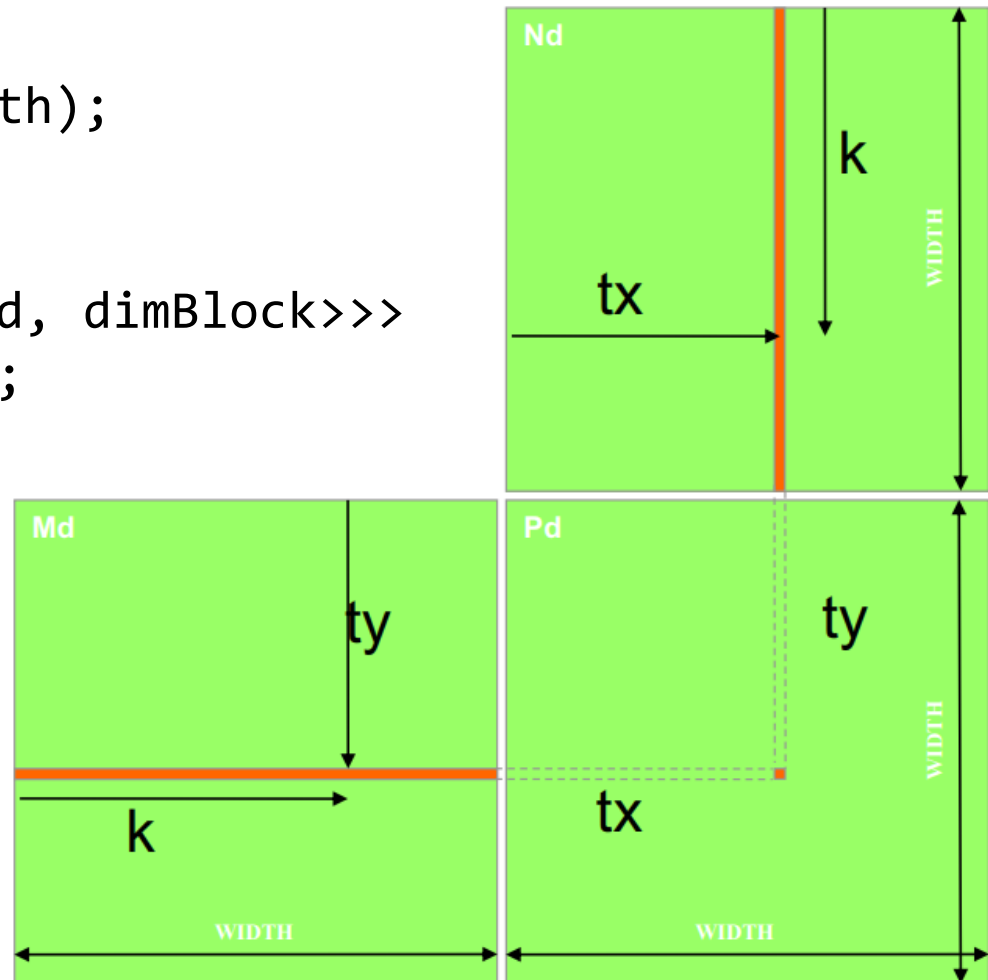
Primer: Množenje matrica (6)

// Konfiguracija kernela

```
dim3 dimGrid(1, 1);  
dim3 dimBlock(Width, Width);
```

// Pokretanje

```
MatrixMulKernel<<<dimGrid, dimBlock>>>  
    (Md, Nd, Pd, Width);
```



Primer: Množenje matrica (5)

KERNEL:

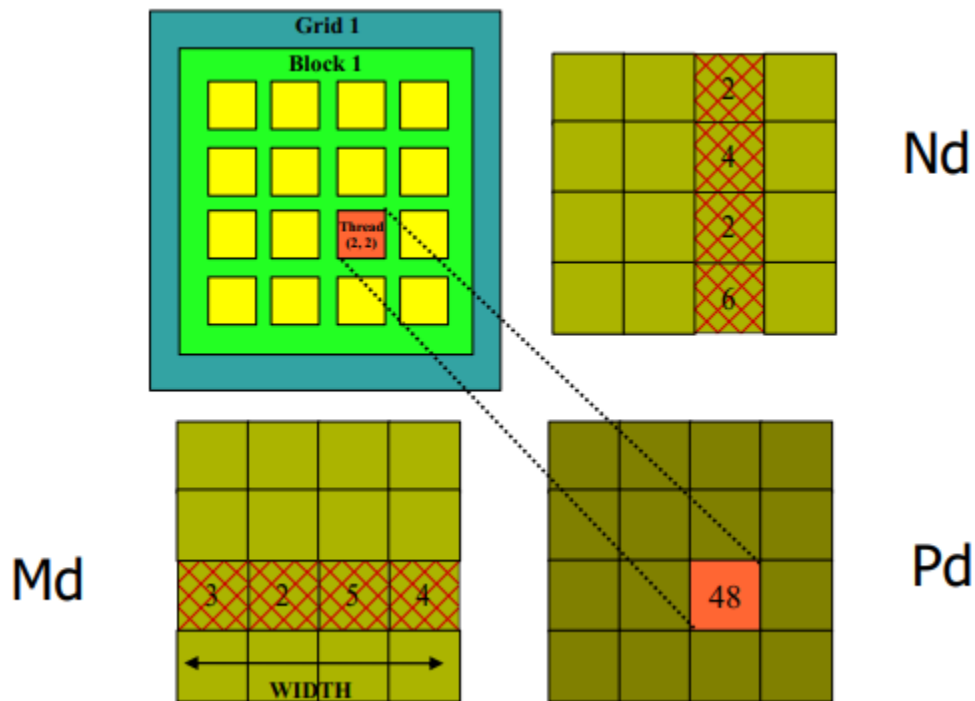
```
__global__ void MatrixMulKernel(float* Md, float* Nd, float* Pd, int
Width)
{
    // Pvalue is used to store the element of the matrix
    // that is computed by the thread
    float Pvalue = 0;

    for (int k = 0; k < Width; ++k)
    {
        float Melement = Md[threadIdx.y * Width + k];
        float Nelement = Nd[k * Width + threadIdx.x];
        Pvalue += Melement * Nelement;
    }

    Pd[threadIdx.y * Width + threadIdx.x] = Pvalue;
}
```

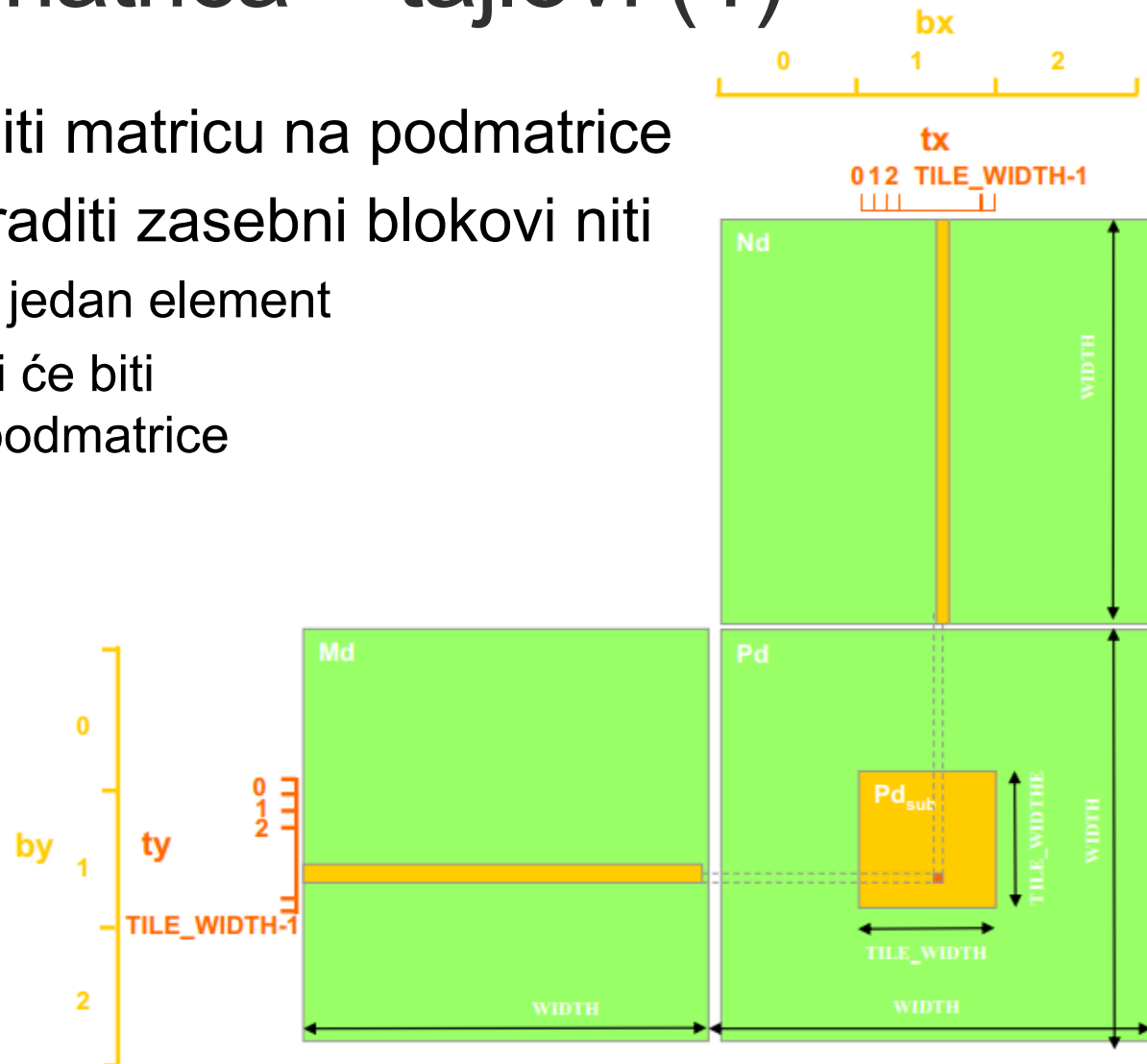
Primer: Množenje matrica - Nedostaci (1)

- Koristi se samo jedan blok niti
 - Matrice mogu biti samo ograničene veličine



Množenje matrica – tajlovi (1)

- Rešenje – podeliti matricu na podmatrice (tiles) koje će obraditi zasebni blokovi niti
 - Svaka nit računa jedan element
 - Veličina bloka niti će biti jednaka veličini podmatrice



Množenje matrica – tajlovi (2)

- Primer množenja matrice 4x4, korišćenjem blokova niti dimenzija 2x2 niti

Blok(0,0)

Blok(1,0)

$P_{0,0}$	$P_{1,0}$	$P_{2,0}$	$P_{3,0}$
$P_{0,1}$	$P_{1,1}$	$P_{2,1}$	$P_{3,1}$
$P_{0,2}$	$P_{1,2}$	$P_{2,2}$	$P_{3,2}$
$P_{0,3}$	$P_{1,3}$	$P_{2,3}$	$P_{3,3}$

Blok(0,1)

Blok(1,1)

$Md_{0,0}$	$Md_{1,0}$	$Md_{2,0}$	$Md_{3,0}$
$Md_{0,1}$	$Md_{1,1}$	$Md_{2,1}$	$Md_{3,1}$

$Nd_{0,0}$	$Nd_{1,0}$		
$Nd_{0,1}$	$Nd_{1,1}$		
$Nd_{0,2}$	$Nd_{1,2}$		
$Nd_{0,3}$	$Nd_{1,3}$		

$Pd_{0,0}$	$Pd_{1,0}$	$Pd_{2,0}$	$Pd_{3,0}$
$Pd_{0,1}$	$Pd_{1,1}$	$Pd_{2,1}$	$Pd_{3,1}$
$Pd_{0,2}$	$Pd_{1,2}$	$Pd_{2,2}$	$Pd_{3,2}$
$Pd_{0,3}$	$Pd_{1,3}$	$Pd_{2,3}$	$Pd_{3,3}$

Množenje matrica – tajlovi (3)

KERNEL:

```
__global__ void MatrixMulKernel
(float* Md, float* Nd, float* Pd, int Width)
{
    // Računanje indeksa za vrstu za Pd i M
    int Row = blockIdx.y * TILE_WIDTH + threadIdx.y;
    // Računanje indeksa za kolonu za Pd i N
    int Col = blockIdx.x * TILE_WIDTH + threadIdx.x;
    float Pvalue = 0;

    // Svaka nit računa jedan element za podmatricu bloka
    for (int k = 0; k < Width; ++k)
        Pvalue += Md[Row * Width + k] * Nd[k * Width + Col];

    Pd[Row * Width + Col] = Pvalue;
}
```

Množenje matrica – tajlovi (4)

- Poziv kernela?

Množenje matrica – tajlovi (4)

- Poziv kernela?

```
// Setup the execution configuration
dim3 dimGrid(Width/TILE_WIDTH, Width/TILE_WIDTH);
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);

// Launch the device computation threads!
MatrixMulKernel<<<dimGrid, dimBlock>>>(Md, Nd, Pd, Width);
```

Množenje matrica - deljiva memorija (1)

- Jednostavni CUDA kerneli koji obezbeđuju da niti pristupaju određenim delovima podataka, koji su prethodno prebačeni na uređaj.
- Latenca globalne memorije i ograničen bandwidth za pristup dovode do zagušenja
- Rešenje: dodatni metodi za pristup memoriji (različiti tipovi memorije)

Množenje matrica - deljiva memorija (2)

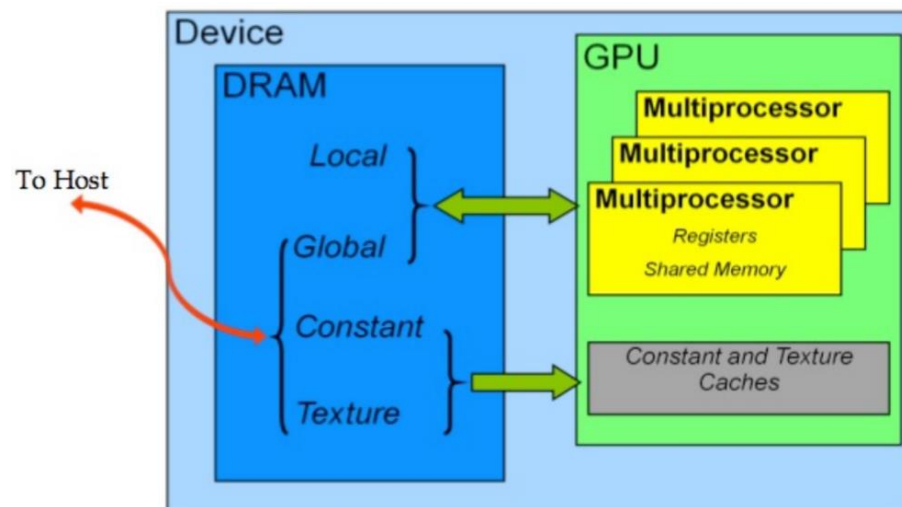
- Najbitniji deo kernela jeste for petlja koja se koristi da se računaju skalarni proizvodi.
 - U svakoj iteraciji ove petlje, imamo dva pristupa globalnoj memoriji, jedno FP množenje i jedno FP sabiranje.
 - Odnos FP operacija i pristupa globalnoj memoriji - 1:1.
 - **CGMA** odnos predstavlja broj FP operacija koji se izvršavaju za svaki pristup globalnoj memoriji u jednom delu (regionu) CUDA programa.

Množenje matrica - deljiva memorija (3)

- Sa CGMA odnosom koji je 1:1, kernel koji se koristi za množenje matrica će izvršavati ne više od 21.6 **biliona** FP operacija u sekundi (gigaflopsi) (kartica G80, global memory access bandwidth od 86.4GB/s)
- Svaka FP operacija zahteva jedan single-precision global memory datum.
- Maksimalni procenjeni broj operacija za istu karticu je 367gflops.

CUDA device memory types - podsetnik

Memory	Location on/off chip	Cached	Access	Scope	Lifetime
Register	On	n/a	R/W	1 thread	Thread
Local	Off	†	R/W	1 thread	Thread
Shared	On	n/a	R/W	All threads in block	Block
Global	Off	†	R/W	All threads + host	Host allocation
Constant	Off	Yes	R	All threads + host	Host allocation
Texture	Off	Yes	R	All threads + host	Host allocation

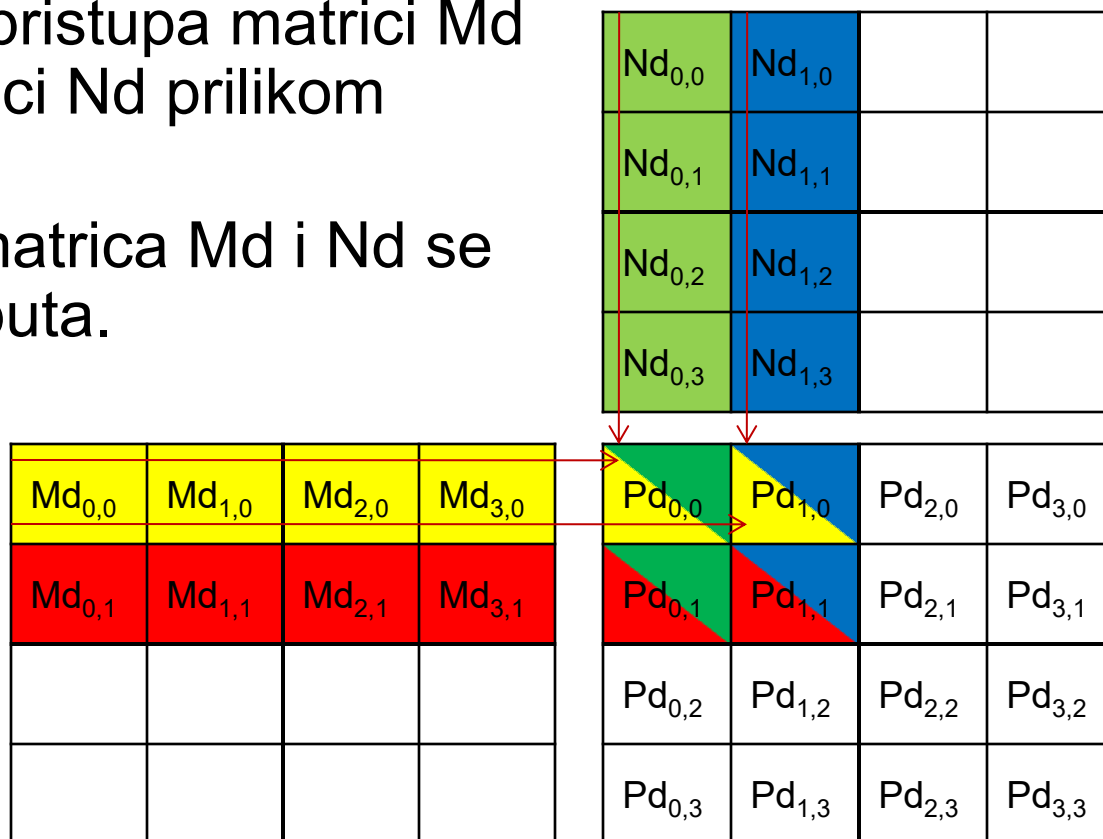


Strategija za redukciju broja pristupa globalnoj memoriji

- Partitionisanje podataka u podskupove, ili tajlove (eng. *tiles*)
- Cilj je da svaki tajl može da se smesti u deljivu memoriju
- Globalna memorija (Global memory-GM) je velikog kapaciteta, ali spora, dok je deljiva memorija (Shared memory-SHM) mala i brza
- Problem: ne mogu sve strukture podataka da se podele u tajlove.

Množenje matrica - deljiva memorija (1)

- Svaka nit ima četiri pristupa matrici Md i četiri pristupa matrici Nd prilikom izvršenja.
- Svakom elementu matrica Md i Nd se pristupa tačno dva puta.



Množenje matrica - deljiva memorija (2)

- **Šta ako bi niti koje pristupaju istim elementima mogle međusobno da sarađuju?**
 - Elementi se mogu iz globalne memorije učitati samo jednom, što bi smanjilo ukupan broj pristupa globalnoj memoriji.
 - Svakom elementu matrica M_d i N_d se pristupa tačno dva puta.
 - Potencijalno možemo smanjiti ukupan broj pristupa globalnoj memoriji
 - Redukcija je proporcionalna broju blokova: sa $N \times N$ blokova, potencijalna redukcija je N puta.

Množenje matrica - deljiva memorija (3)

PRISTUP GLOBALNOJ MEMORIJI – NITI IZ BLOKA (0,0)

Access order ↓

$Pd_{0,0}$ Thread(0,0)	$Pd_{1,0}$ Thread(1,0)	$Pd_{0,1}$ Thread(0,1)	$Pd_{1,1}$ Thread(1,1)
$Md_{0,0} * Nd_{0,0}$	$Md_{0,0} * Nd_{1,0}$	$Md_{0,1} * Nd_{0,0}$	$Md_{0,1} * Nd_{1,0}$
$Md_{1,0} * Nd_{0,1}$	$Md_{1,0} * Nd_{1,1}$	$Md_{1,1} * Nd_{0,1}$	$Md_{1,1} * Nd_{1,1}$
$Md_{2,0} * Nd_{0,2}$	$Md_{2,0} * Nd_{1,2}$	$Md_{2,1} * Nd_{0,2}$	$Md_{2,1} * Nd_{1,2}$
$Md_{3,0} * Nd_{0,3}$	$Md_{3,0} * Nd_{1,3}$	$Md_{3,1} * Nd_{0,3}$	$Md_{3,1} * Nd_{1,3}$

Množenje matrica - deljiva memorija (4)

- **Kako niti mogu biti organizovane tako da se redukuje pristup memoriji?**
- Sve niti kolaborativno učitavaju elemente M_d i N_d matrice u deljivu memoriju pre nego što ih koriste u svojim izračunavanjima (računanje skalarnog proizvoda).
 - Potencijalni problem: veličina deljive memorije
 - Rešenje: M_d i N_d matrice se dele u tajlove. Veličina tajlova određuje se količinom dostupne deljive memorije.
- Svaki skalarni proizvod se deli u faze
 - U svakoj fazi, sve niti u bloku kolaborativno učitavaju jedan tajl matrice M_d i jedan tajl matrice N_d u deljivu memoriju (**M_{ds} , N_{ds}**). Nakon učitavanja, ove se vrednosti koriste za skalarni proizvod.
- Ako je ulazna matrica dimenzije N , i veličina tajlova je $TILE_WIDTH$, skalarni proizvod se obavlja u $N/TILE_WIDTH$ faza.

Množenje matrica - deljiva memorija (5)

PRVA FAZA, SVI BLOKOVI

$B_{0,0}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

$B_{0,1}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

$B_{1,0}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

$B_{1,1}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

Množenje matrica - deljiva memorija (6)

DRUGA FAZA, SVI BLOKOVİ

$B_{0,0}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

$B_{0,1}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

$B_{1,0}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

$B_{1,1}$

Md 0,0	Md 1,0	Md 2,0	Md 3,0
Md 0,1	Md 1,1	Md 2,1	Md 3,1
Md 0,2	Md 1,2	Md 2,2	Md 3,2
Md 0,3	Md 1,3	Md 2,3	Md 3,3

Nd 0,0	Nd 1,0	Nd 2,0	Nd 3,0
Nd 0,1	Nd 1,1	Nd 2,1	Nd 3,1
Nd 0,2	Nd 1,2	Nd 2,2	Nd 3,2
Nd 0,3	Nd 1,3	Nd 2,3	Nd 3,3

Množenje matrica - deljiva memorija (7)

• BLOK (0,0)

	Phase 1			Phase 2		
$T_{0,0}$	Md_{0,0} ↓ Mds _{0,0}	Nd_{0,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{1,0} *Nds _{0,1}	Md_{2,0} ↓ Mds _{0,0}	Nd_{0,2} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{1,0} *Nds _{0,1}
$T_{1,0}$	Md_{1,0} ↓ Mds _{1,0}	Nd_{1,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{0,0} *Nds _{1,0} + Mds _{1,0} *Nds _{1,1}	Md_{3,0} ↓ Mds _{1,0}	Nd_{1,2} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{0,0} *Nds _{1,0} + Mds _{1,0} *Nds _{1,1}
$T_{0,1}$	Md_{0,1} ↓ Mds _{0,1}	Nd_{0,1} ↓ Nds _{0,1}	PdValue _{0,1} += Mds _{0,1} *Nds _{0,0} + Mds _{1,1} *Nds _{0,1}	Md_{2,1} ↓ Mds _{0,1}	Nd_{0,3} ↓ Nds _{0,1}	PdValue _{0,1} += Mds _{0,1} *Nds _{0,0} + Mds _{1,1} *Nds _{0,1}
$T_{1,1}$	Md_{1,1} ↓ Mds _{1,1}	Nd_{1,1} ↓ Nds _{1,1}	PdValue _{1,1} += Mds _{0,1} *Nds _{1,0} + Mds _{1,1} *Nds _{1,1}	Md_{3,1} ↓ Mds _{1,1}	Nd_{1,3} ↓ Nds _{1,1}	PdValue _{1,1} += Mds _{0,1} *Nds _{1,0} + Mds _{1,1} *Nds _{1,1}

time $\longrightarrow$

Množenje matrica - deljiva memorija (8)

- Mds čuva elemente Md koji su u SHM, Nds čuva elemente Nd koji su u SM
- Na početku faze 1, 4 niti bloka B00 učitavaju elemente u Mds
- T00: $Mds_{00} \leftarrow Md_{00}$
- T10: $Mds_{10} \leftarrow Md_{10}$
- T01: $Mds_{01} \leftarrow Md_{01}$
- T11: $Mds_{11} \leftarrow Md_{11}$
- Na sličan način se učitavaju elementi matrice Nd u Nds
- Zatim se izračunavaju odgovarajući skalarni proizvodi

Množenje matrica - deljiva memorija (9)

- Svaka vrednost u SHM se koristi 2 puta
 - Npr. i T01 i T11 koriste vrednost Mds11
- Na taj način smanjuje se pristup memoriji dva puta
- Izračunavanje svakog vektorskog proizvoda se za ovaj primer odvija u 2 faze.
- U opštem slučaju, za matrice reda N i veličinu tajla TILE_WIDTH, potrebno je **$N/\text{TILE_WIDTH}$** faza
- U svakoj fazi, **isti** Mds i Nds se koriste da čuvaju delove Md i Nd matrica korišćenih u toj fazi
- Ovo omogućava da mala SHM služi za većinu pristupa za koje je korišćena GM

Množenje matrica - deljiva memorija (10)

- To je zato što svaka faza fokusirana na mali podskup elemenata ulazne matrice, što predstavlja lokalnost pri pristupu
- Lokalnost pristupa omogućava korišćenje malih brzih memorija koje opslužuju većinu pristupa i smanjuju pristup sporoj GM
- Lokalnost je važna za postizanje visokih performansi

Množenje matrica - deljiva memorija (11)

POZIV KERNELA

```
// Setup the execution configuration
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
dim3 dimGrid(Width / TILE_WIDTH, Width / TILE_WIDTH);

// Launch the device computation threads!
MatrixMulKernel<<<dimGrid, dimBlock>>> (Md, Nd, Pd, Width);
```

Množenje matrica - deljiva memorija (12)

KERNEL (1)

```
__global__ void MatrixMulKernel
(float* Md, float* Nd, float* Pd, int Width) {

    __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
    __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;
    //Identify the row and column of the
    //Pd element to work on
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;
    float Pvalue = 0;
```

Množenje matrica - deljiva memorija (13)

KERNEL (2)

```
// Loop over the Md and Nd tiles required to compute
//the Pd element
for (int m = 0; m < Width/TILE_WIDTH; ++m) {
    // Collaborative loading of Md and Nd tiles
    // into shared memory
    Mds[ty][tx] = Md[Row*Width + (m*TILE_WIDTH + tx)];
    Nds[ty][tx] = Nd[Col + (m*TILE_WIDTH + ty)*Width];
    __syncthreads();

    for (int k = 0; k < TILE_WIDTH; ++k)
        Pvalue += Mds[ty][k] * Nds[k][tx];
    __syncthreads();
}
Pd[Row*Width+Col] = Pvalue;
}
```

Množenje matrica - deljiva memorija (14)

- *Mds* i *Nds* su promenljive u SHM i njihova oblast važenja su niti u okviru bloka
- Promenljive *bx*, *by*, *tx* i *ty* su automatske skalarne promenljive i smeštene su u registrima.
 - Njihova oblast važenja je nit i postoji privatna kopija za svaku nit
- Promenljive *Row* i *Col* su indeksi vrste i kolone rezultujućeg Pd elementa
- *Pvalue* je promenljiva u kojoj se čuvaju privremene vrednosti rezultujućeg elementa PD

Množenje matrica - deljiva memorija (15)

- U svakoj fazi izračunavanja, svaka nit u nekom bloku učitava po jedan element matrice M_d u SHM
- $TILE_WIDTH^2$ niti sarađuju u učitavanju $TILE_WIDTH^2$ različitih vrednosti matrice M_d u SM
- Slično važi i za elemente matrice N_d
- Da bi se obezbedilo da sve niti u okviru istog bloka matrice kompletiraju učitavanje vrednosti u M_d s i N_d s pre izvođenja izračunavanja dela skalarnog proizvoda koristi se poziv ***syncthreads()***

Množenje matrica - deljiva memorija (20)

- Nakon toga, moguće je otpočeti izračunavanje gde svaka nit u K petlji izvodi izračunavanje jednog dela svog skalarnog proizvoda
- Nakon izvršenja K petlje, neophodan je još jedan poziv ***syncthreads()*** funkcije koja sad obezbeđuje da su sve niti u okviru nekog bloka završile sa korišćenjem sadržaja Mds i Nds pre nego što započne sledeća faza i učitaju se novi elementi u Mds i Nds

Množenje matrica-performanse

- Prilikom opisanog množenja matrica, pristip GM je smanjen `TILE_WIDTH` puta
 - Ako se koriste 16x16 blokovi, pristup se smanjuje 16 puta
- Prednosti korišćenja deljene memorije:
 - Svaki blok niti treba da ima veliki broj niti
 - `TILE_WIDTH` ne bi trebalo da bude manje od 16
 - $16 \times 16 = 256$ niti po bloku
 - Treba da bude veliki broj blokova niti
 - Rezultujuća matrica veličine 1024×1024 daje $64 \times 64 = 4096$ blokova niti

Množenje matrica-performanse

- U takvom slučaju, svaki blok niti ima $2 \times 256 = 512$ pristupa (čitavanja) iz globalne memorije
- Zatim se vrši $256 \times (2 \times 16) = 8192$ operacija množenja i sabiranja
- U takvom slučaju memorijski propusni opseg više nije limitirajući faktor
- **Broj računskih operacija mnogo veći od broja pristupa memoriji**
 - $8192 \gg 512$

Atomične operacije

Atomične operacije

- $x++$
- Čitanje vrednosti x
- Inkrementiranje pročitane vrednosti
- Upis rezultata u x

read-modify-write operacija

Atomične operacije – dve niti

$X = 7$, Niti A i B inkrementiraju X

Korak	Primer
1. Nit A čita x	A čita vrednost 7
2. Nit A inkrementira pročitano vrednost	A računa 8
3. A upisuje rezultat u x	$x \leftarrow 8$
4. Nit B čita x	B čita vrednost 8
5. Nit B inkrementira pročitano vrednost	B računa 9
6. B upisuje rezultat u x	$x \leftarrow 9$

Korak	Primer
1. Nit A čita x	A čita vrednost 7
2. Nit B čita x	B čita vrednost 7
2. Nit A inkrementira pročitano vrednost	A računa 8
5. Nit B inkrementira pročitano vrednost	B računa 8
3. A upisuje rezultat u x	$x \leftarrow 8$
6. B upisuje rezultat u x	$x \leftarrow 8$

Compute capability

- Sve operacije koje smo dosad naveli su podržane od strane svake CUDA-capable grafičke kartice
- Kao što postoje različiti setovi instrukcija za CPU (MMX, SSE, SSE2), postoje različiti setovi instrukcija i za GPU
- NVIDIA funkcionalnosti i mogućnosti grafičkih kartica obeležava sa ***compute capability***
- Na primer, atomične operacije su podržane tek od *compute capability* 1.1 pa naviše

Compute capability - kompajliranje

- Da bi se kompajlirao CUDA kod, potrebno je naznačiti koji je minimalni compute capability.
- Navođenjem istog, kompajleru se daje mogućnost i za dodatne optimizacije.

`nvcc -arch=sm_11`

Race conditions

- Hazard koji se javlja kada rezultat izvršenja zavisi od trenutka u kom se dešavaju događaji nad kojima nemamo kontrolu
 - Redosled izvršenja niti
- Problemi nastaju kada se naruši SIMD paradigma
- Atomične operacije su *brute force* način da se ovi problemi zaobiđu
 - Atomics, locks, and mutex

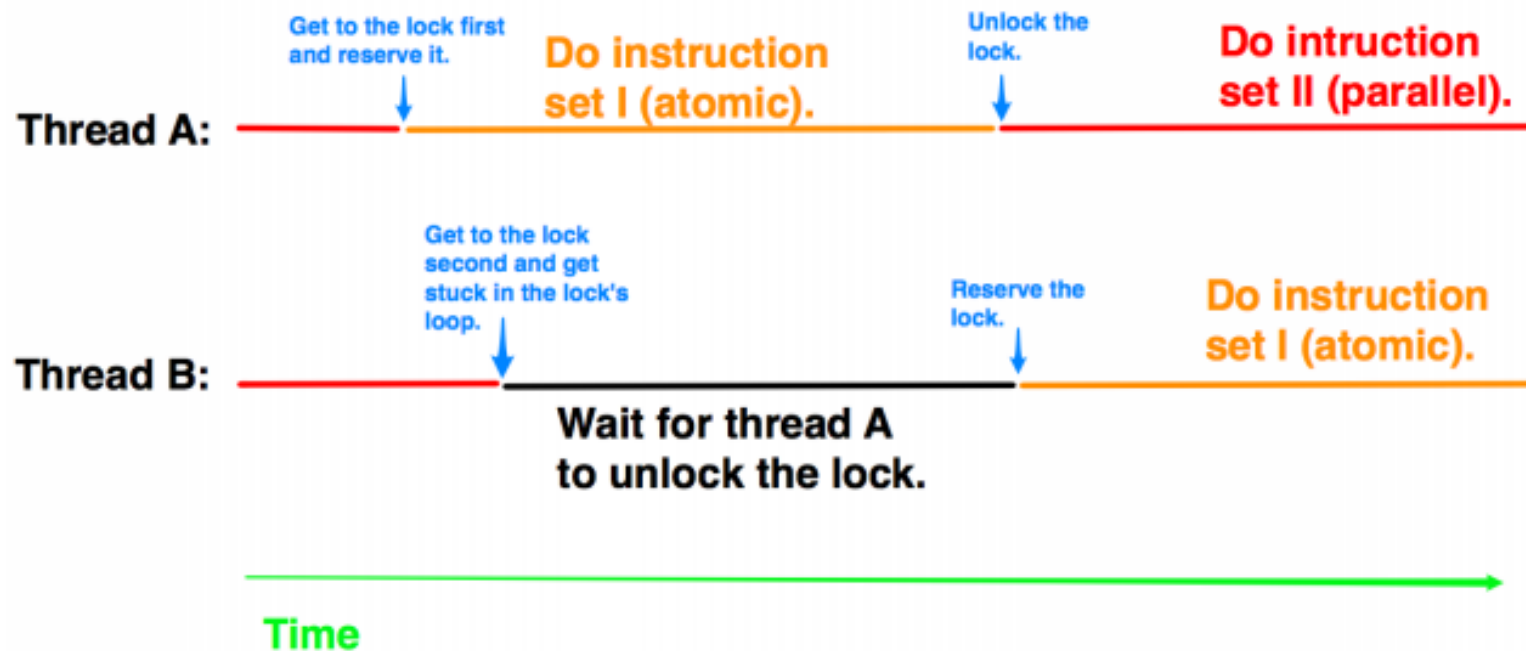
Atomici

- `atomicAdd()`
- `atomicSub()`
- `atomicMin()`
- `atomicMax()`
- `atomicInc()`
- `atomicDec()`
- `atomicExch()`
- `atomicCAS()`
- `atomicAnd()`
- `atomicOr()`
- `atomicXor()`

Locks and mutex

- Lock:
 - Mehanizam u paralelnom programiranju kojim se obezbeđuje da ceo segment koda bude izvršen atomično
- Mutex
 - - “mutual exclusion”, princip koji stoji iza lock-ova.
 - Dok jedna nit izvršava deo koda unutar lock-a, mutex se drži zaključanim tako da nijedna druga nit ne može da ga otključa

Locks and mutex



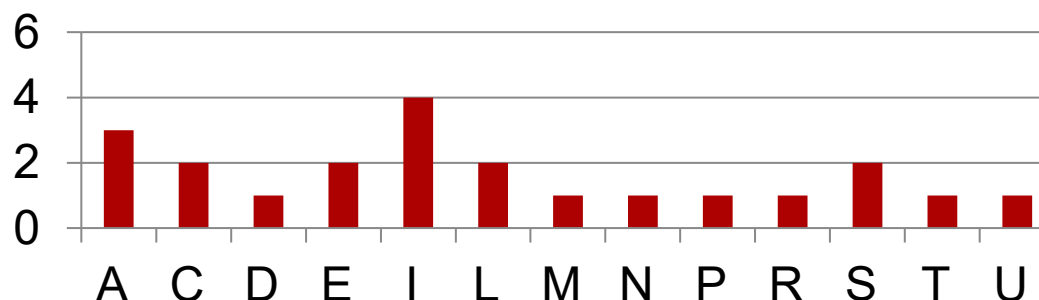
Primer: Računanje histograma

- Jedan od čestih zadataka jeste računanje histograma za dati set podataka
- Histogramom se predstavlja broj pojavljivanja svakog elementa u datom skupu
- Primer:

Skup podataka: *Paralelni sistemi i CUDA C*

Histogram:

3	2	1	2	4	2	1	1	1	1	2	1	1
A	C	D	E	I	L	M	N	P	R	S	T	U



Računanje histograma – CPU (1)

```
#define SIZE (100*1024*1024)

int main(void)
{
    //kreiranje random niza
    unsigned char *buffer = (unsigned char*)
                           big_random_block(SIZE);
    //kreiranje histograma i inicijalizacija na 0
    unsigned int histo[256];
    for(int i=0; i<256; i++)
        histo[i] = 0;

    ...
}
```

Računanje histograma – CPU (2)

...

```
// ideja je da kad god vidimo neku vrednost z u
// nizu, inkrementiramo vrednost bina z u histogramu. Na taj
// način brojimo broj ponavljanja elementa z u nizu
    for (int i=0; i<SIZE; i++)
        histo[buffer[i]]++;
// provera tačnosti
    long histoCount = 0;
    for (int i=0; i<256; i++)
    {
        histoCount += histo[i];
    }

    printf("Histogram Sum: %ld\n", histoCount);

    free(buffer);
    return 0;
}
```

Računanje histograma – GPU (1)

```
#define SIZE (100*1024*1024)

int main(void)
{
    //kreiranje random niza
    unsigned char *buffer = (unsigned char*)
        big_random_block(SIZE);

    // alociranje memorije na GPU i prenos podataka
    unsigned char *dev_buffer;
    unsigned int *dev_histo;
    HANDLE_ERROR(cudaMalloc((void**)&dev_buffer, SIZE));
    HANDLE_ERROR(cudaMemcpy(dev_buffer,buffer,SIZE,cudaMemcpyHostToDevice));
    HANDLE_ERROR(cudaMalloc( (void**)&dev_histo, 256 * sizeof(long)));

    HANDLE_ERROR(cudaMemset(dev_histo, 0, 256 * sizeof(int)));

    ...
}
```

Računanje histograma – GPU (2)

```
...
    cudaDeviceProp prop;
    HANDLE_ERROR( cudaGetDeviceProperties( &prop, 0 ) );
    int blocks = prop.multiProcessorCount;
    histo_kernel<<<blocks*2,256>>>( dev_buffer, SIZE, dev_histo );

    unsigned int histo[256];
    HANDLE_ERROR( cudaMemcpy(histo, dev_histo, 256 *
        sizeof(int),cudaMemcpyDeviceToHost));

    long histoCount = 0;
    for (int i=0; i<256; i++)
        histoCount += histo[i];
    printf( "Histogram Sum: %ld\n", histoCount );
    //TODO> provera tačnosti
    cudaFree(dev_histo);
    cudaFree(dev_buffer);
    free(buffer);
    return 0;
}
```

Računanje histograma – GPU (2)

GLOBALNA MEMORIJA

```
#define SIZE (100*1024*1024)
__global__ void histo_kernel(unsigned char *buffer,
                             long size,
                             unsigned int *histo)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    while (i < size)
    {
        atomicAdd( &(histo[buffer[i]]), 1 );
        i += stride;
    }
}
```

Računanje histograma – GPU (2)

GLOBALNA I DELJIVA MEMORIJA

```
__global__ void histo_kernel(unsigned char *buffer,
                             long size,
                             unsigned int *histo)
{
    __shared__ unsigned int temp[256];
    temp[threadIdx.x] = 0;
    __syncthreads(); // inicijalizacija na 0

    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int offset = blockDim.x * gridDim.x;

    while (i < size)
    {
        atomicAdd( &temp[buffer[i]], 1);
        i += offset;
    }

    __syncthreads();
    atomicAdd( &(histo[threadIdx.x]), temp[threadIdx.x] );
}
```